Recovering 2-layer ReLU Networks: Exact Recovery Algorithms, ICA Initialisation and Local Geometry

Sivaram Swaminathan

Supervisor: Dr. Jean Honorio

Semester 2, 2026

Abstract

Recovering the weights of a 2 layer ReLU neural network from input-output samples is a nonconvex problem for which provable exact recovery algorithms and practical algorithms behave differently. Bakshi et al. [2019] give a polynomial time algorithm that combines independent component analysis (ICA) with combinatorial sparsity-pattern recovery to recover the weights exactly. Gradient descent carries no such guarantee, and local spurious minima are known to exist in similar models [Safran and Shamir, 2018]. This thesis asks where that gap comes from, and what closes it. We provide a practical implementation of the algorithm outlined in [Bakshi et al., 2019] and find that it shows poor recovery under finite sample regimes, and catastrophic degradation when exposed to error and contamination. To explain this behaviour we diagnose the source of error in Bakshi et al. [2019]'s algorithm as a key threshold described without explicit constants, and provide a heuristic to optimise it based on asymmetrical effect of different error types. In spite of the inaccuracy, we find that the algorithm is an excellent initialiser: ICA-initialised gradient descent reaches machine precision more than 4 times as fast as random or spectral initialisations, because it recovers the orientation of each hidden unit rather than just their subspace. We attempt to study the local geometry of the loss which contributes to this, by deriving the Hessian of the loss, its Gauss-Newton decomposition, and characterising the conditions under which it is benign with a computable certificate. Further, we show that the kernel of the population Gauss-Newton matrix is *exactly* the tangent space of the positive-rescaling symmetry whenever the weight matrices have full rank. We then provide a bound of the minimum eigenvalue of the empirical Hessian in terms of that of the population. Future work can refine the analysis of the local geometry of the loss near ground truth to determine if ICA-based initialisation can land in a benign basin. Ultimately, exact combinatorial recovery is fragile at finite sample sizes, gradient methods are robust yet globally unguaranteed, but the effective combination uses the former to place the latter within favourable convergence conditions.

1 Introduction

Many optimisation problems in neural networks are nonconvex, and current methods of solving such problems involve gradient descent methods which can converge to local optima as opposed to global ones. Up till now, the only provable convergence theorem closest to our setting with empirical backing is that of Zhong et al. [2017], for networks of the form

$$y = \sum_{i=1}^{k} \mathrm{v}_i^* \cdot \phi(\mathrm{w}_i^{*\top} x),\ \mathrm{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

This is a 1 hidden layer network, with the final layer being a k dimensional vector consisting of the v_i^*. In a similar problem with each $\mathrm{v}_i^* := 1$, Safran and Shamir [2018] prove that there exist spurious local minima. It is the case that networks of greater depth and complexity are trained by local search on nonconvex losses, and while working empirically, theory lags behind with no convergence guarantee.

A prominent example is a 2-layer rectified neural network of the form

$$\mathbf{A} = \mathbf{U}^* \sigma(\mathbf{V}^* X),$$

where $\mathbf{X} \in \mathbb{R}^{d \times n}$, $\mathbf{X}_{:,j} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$ represents the (Gaussian) input samples, $\mathbf{A}$ represents the respective outputs, and σ is the element-wise ReLU activation function, $\sigma(\mathrm{x}) := \max(0, x)$. The objective boils down to finding the weight matrices $\mathbf{U}^* \in \mathbb{R}^{m \times k}$ and $\mathbf{V}^* \in \mathbb{R}^{k \times d}$, with respect to which, the problem is nonconvex.

Within the constraints of such a 2-layer ReLU network, there exist polynomial time algorithms which achieve exact recovery of said weight matrices, up to arbitrary permutation and scaling [Bakshi et al., 2019], avoiding gradient descent methods. The method utilises tensor decomposition for general $\mathbf{V}^*$ and Independent Component Analysis (ICA) for orthonormal $\mathbf{V}^*$. To the best of our knowledge, this problem structure is the simplest network where a provably exact method (without GD) and standard GD methods both exist for random samples, allowing us to compare the two.

Where Bakshi et al. [2019] fails to provide empirical backings for their algorithms, we aim to implement and characterise the performance with finite n samples, and with error and contamination. We also further analyse the GD objective and see if a local convergence guarantee may hold. This regime acts as a natural extension to Zhong et al. [2017]'s case, where instead of $v^* \in \mathbb{R}^k$, we have $\mathbf{U}^* \in \mathbb{R}^{m \times k}$, trained alongside $\mathbf{V}^*$ (replacing their w_i^*'s).

Our contributions are as follows.

- **Finite-sample diagnosis.** We give, to the best of our knowledge, the first finite-sample implementation of the orthonormal case of Bakshi et al. [2019]. Given true support it is exact, else its error comes from the thresholding

step with false positive being asymmetrically costlier than false negatives (§3.1.3–3.1.4).

- **A principled threshold.** Modelling ICA output as activation plus Gaussian noise gives closed forms for both error rates, and the smallest threshold that excludes false positives, $\tau^* = \hat{\sigma}\,\Phi^{-1}(1 - 2/(k\ell))$, approximates the optimal threshold without using the ground truth, thereby bridging the gap between Bakshi et al. [2019]'s theoretical paradigm and practical implementation (§3.1.4).
- **ICA as an initialiser.** Under noise and contamination the algebraic method degrades sharply while gradient descent does not, but ICA-initialised gradient descent reaches machine precision up to roughly 4 times faster than random or spectral initialisation as it resolves the orientation of hidden units (§3.2).
- **Local geometry.** We derive the Hessian in closed form, prove that the kernel of the population Gauss–Newton matrix is exactly the rescaling symmetry, bound the empirical curvature at the truth, and give a computable curvature certificate which the ICA initialisation has been observed to satisfy (§3.3).

Future work entails optimising ICA with respect to problem specific constraints, and possibly establishing a basin of positive curvature modulo symmetry, enabling exact recovery for end-to-end ICA initialisation + gradient descent.

2 Preliminaries

2.1 Notation

Bold upper-case letters denote matrices and lower-case letters denote vectors; $[n] = \{1, \ldots, n\}$. For a matrix $\mathbf{M}$, $\mathbf{M}_{:,j}$ is its j-th column, $\mathbf{M}^\dagger$ its Moore–Penrose pseudo-inverse, $\|\mathbf{M}\|_2$ its spectral norm and $\|\mathbf{M}\|_F$ its Frobenius norm. For symmetric $\mathbf{M}$, $\lambda_{\min}(\mathbf{M})$ and $\lambda_{\max}(\mathbf{M})$ denote its smallest and largest eigenvalues. $\operatorname{vec}(\mathbf{M})$ stacks the columns of $\mathbf{M}$ into a vector, $\operatorname{diag}(\mathrm{v})$ is the diagonal matrix with diagonal v, $\otimes$ is the Kronecker product and $\odot$ the entry-wise (Hadamard) product. The commutation matrix $\mathbf{K}_{a,b}$ satisfies $\mathbf{K}_{a,b}\operatorname{vec}(\mathbf{M}) = \operatorname{vec}\left(\mathbf{M}^\top\right)$ for $\mathbf{M} \in \mathbb{R}^{a\times b}$. We use the identities [Petersen and Pedersen, 2012]

$$\operatorname{vec}(\mathbf{AXB}) = (\mathbf{B}^\top \otimes \mathbf{A})\operatorname{vec}(\mathbf{X}),$$

$$(\mathbf{A}\otimes\mathbf{B})(\mathbf{C}\otimes\mathbf{D}) = (\mathbf{AC})\otimes(\mathbf{BD}),$$

$$\operatorname{vec}(\mathbf{A}\odot\mathbf{B}) = \operatorname{diag}(\operatorname{vec}(\mathbf{A}))\operatorname{vec}(\mathbf{B}).$$

$\sigma(t) = \max(t, 0)$ is the entry-wise ReLU and $\mathbf{1}(\cdot)$ is the entry-wise indicator, so that $\sigma(t) = \mathbf{1}(t > 0)\,t$.

2.2 Problem Setup

The noiseless model is given by

$$\mathbf{A} = \mathbf{U}^* \sigma(\mathbf{V}^* \mathbf{X}),$$

where $\mathbf{A} \in \mathbb{R}^{m \times n}$ are the outputs and $\mathbf{X} \in \mathbb{R}^{d \times n}$ the inputs, whose columns $\mathrm{x}_j \coloneqq \mathbf{X}_{:,j} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$ are i.i.d. Assume that $\mathbf{U}^* \in \mathbb{R}^{m \times k}$ has full column rank and $\mathbf{V}^* \in \mathbb{R}^{k \times d}$ has full row rank; for Algorithm 1 and Algorithm 2 assume also $\mathbf{V}^*$ has orthonormal rows.

To model additive noise, we perturb the outputs as

$$\mathbf{A} = \mathbf{U}^* \sigma(\mathbf{V}^* \mathbf{X}) + \boldsymbol{\mathcal{E}}, \qquad \mathcal{E}_{ij} \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, p^2),$$

where p is the noise level. To model contamination, we loosely follow the setup of Bulusu et al. [2022], a fixed-fraction version of Huber's contamination model: a subset $\mathcal{C} \subset [n]$ of $\lfloor q \cdot n \rceil$ columns is chosen uniformly at random, and each column $j \in \mathcal{C}$ is replaced by an outlier $\mathrm{c}_j \sim \mathcal{N}(5 \cdot \mathbf{1}_m,\ \mathbf{I}_m)$. Thus

$$\mathbf{A}_{:,j} = \begin{cases} \mathbf{U}^* \sigma(\mathbf{V}^* \mathrm{x}_j) + \boldsymbol{\mathcal{E}}_{:,j}, & j \notin \mathcal{C}, \\ \mathrm{c}_j, & j \in \mathcal{C}. \end{cases}$$

The noiseless model corresponds to $p = q = 0$. Hereon, p and q refer to the noise level and the contamination fraction respectively.

Observe the weights can only be recovered up to a permutation and positive rescaling of the hidden units. For any permutation matrix $\mathbf{\Pi}$ and diagonal matrix $\mathbf{\Lambda}$ with positive diagonal entries, $(\mathbf{U}^* \mathbf{\Pi}^\top \mathbf{\Lambda}^{-1},\ \mathbf{\Lambda \Pi V}^*)$ yields identical $\mathbf{A}$.

Denote the loss of the gradient descent objective as

$$f(\mathbf{U}, \mathbf{V}) = \|\mathbf{U}\, \sigma(\mathbf{V}\mathbf{X}) - \mathbf{A}\|_F^2,$$

jointly nonconvex in $(\mathbf{U}, \mathbf{V})$.

To measure parameter recovery error, we must first align estimates $(\hat{\mathbf{U}}, \hat{\mathbf{V}})$ to the ground truth. We first resolve the scaling ambiguity by mapping $\hat{\mathbf{V}}_{i,:} \overset{\text{def}}{=} \mathrm{v}_i \mapsto \frac{\mathrm{v}_i}{\|\mathrm{v}_i\|},\ \hat{\mathbf{U}}_{:,i} \overset{\text{def}}{=} \mathrm{u}_i \mapsto \|\mathrm{v}_i\| \cdot \mathrm{u}_i;\ \forall\, i \le k$. The permutation is rectified using the Hungarian algorithm [Kuhn, 1955], matching estimated parameter vectors to ground truth by minimising joint normalised squared distance

$$\frac{\|\mathrm{v}_i - \mathrm{v}_j^*\|^2}{\|\mathbf{V}^*\|_F^2} + \frac{\|\mathrm{u}_i - \mathrm{u}_j^*\|^2}{\|\mathbf{U}^*\|_F^2}.$$

We report relative Frobenius errors

$$\varepsilon_{(\hat{\mathbf{U}}, \hat{\mathbf{V}})} = \Big(\frac{\|\hat{\mathbf{U}} - \mathbf{U}^*\|_F}{\|\mathbf{U}^*\|_F},\ \frac{\|\hat{\mathbf{V}} - \mathbf{V}^*\|_F}{\|\mathbf{V}^*\|_F} \Big),$$

computed *after* alignment. We will often omit the recovery metrics for $\hat{\mathbf{U}}$ as it is solved for directly once $\hat{\mathbf{V}}$ is estimated (see §2.4).

2.3 Independent Component Analysis

Consider the model $\mathbf{Y} = \mathbf{MS}$, where $\mathbf{M} \in \mathbb{R}^{k\times k}$ is an unknown invertible mixing matrix and the rows of $\mathbf{S} \in \mathbb{R}^{k\times n}$ are independent, non-Gaussian sources. Independent component analysis (ICA) aims to recover $\mathbf{M}$ and $\mathbf{S}$ from $\mathbf{Y}$ alone. Both are identifiable up to permutation and scaling, provided at most one source is Gaussian [Comon, 1994]. Since a mixture of independent sources is closer to Gaussian than the sources themselves, ICA searches for directions w in which $\mathrm{w}^\top\mathbf{Y}$ is as non-Gaussian as possible. In Section 2.4, ICA is applied with $\mathbf{Y} = \mathbf{TA}$ and sources $\mathbf{S} = \sigma(\mathbf{V}^*\mathbf{X})$.

Bakshi et al. [2019] pose ICA as recovering an estimate $\hat{\mathbf{M}}$ with $\|\hat{\mathbf{M}} - \mathbf{M}\|_F \le \epsilon$, and use the provable algorithm of Arora et al. [2012] as a black box. That algorithm is analysed for sources whose fourth cumulant is negative, whereas rectified Gaussian sources have a positive fourth cumulant; Bakshi et al. [2019] state that the analysis extends to their setting, citing a personal communication.

Our implementation. We are not aware of a public implementation of the algorithm of Arora et al. [2012], which is designed for theoretical guarantees rather than practical use. We therefore use FastICA [Hyvärinen, 1999], which whitens $\mathbf{Y}$ and then maximises an approximation to negentropy. Consequently, our experiments test Algorithm 1with a practical ICA subroutine in place of the provable one, and the finite-sample error of that subroutine is part of what is measured.

2.4 Bakshi et al. [2019]'s Algorithm for Orthonormal $\mathbf{V}^*$

We refer to Algorithm 4 of [Bakshi et al., 2019] as Algorithm 1 throughout

Assume $\mathbf{U}^*$ has full column rank so that the row space of $\mathbf{A} = \mathbf{U}^*\sigma(\mathbf{V}^*\mathbf{X})$ coincides with that of $\sigma(\mathbf{V}^*\mathbf{X})$. Since ReLU maps roughly half of each row of $\sigma(\mathbf{V}^*\mathbf{X})$ to zero, its rows are sparse. Recovering weights reduces to recovering the sparsity pattern of $\sigma(\mathbf{V}^*\mathbf{X})$, restricted to which, $\mathbf{V}^*$ and $\mathbf{U}^*$ tractable via linear systems.

When $\mathbf{V}^*$ has orthonormal rows, $\mathbf{V}^*\mathrm{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_k)$ for $\mathrm{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$, so the k rows of $\sigma(\mathbf{V}^*\mathbf{X})$ are independent and non-Gaussian, as required by ICA; remains that the input has to be such that the mixing matrix is square. Sketching by a Gaussian matrix $\mathbf{T} \in \mathbb{R}^{k\times m}$:

$$\mathbf{TA} = (\mathbf{TU}^*)\,\sigma(\mathbf{V}^*\mathbf{X}),$$

where $\mathbf{TU}^* \in \mathbb{R}^{k\times k}$ invertible with high probability. ICA returns an estimate $\widehat{\mathbf{TU}^*}$ and hence

$$\hat{\mathbf{F}} = \big(\widehat{\mathbf{TU}^*}\big)^{-1}\mathbf{TA} \;\approx\; \sigma(\mathbf{V}^*\mathbf{X}).$$

As $\hat{\mathbf{F}}$ is only approximate, its 'zeros' are not exactly zero, and the sparsity pattern is estimated by thresholding at a constant τ. Full procedure given below.

Algorithm 1 Algorithm 4 of Bakshi et al. [2019] (orthonormal $\mathbf{V}^*$)

Require: Inputs $\mathbf{X} \in \mathbb{R}^{d\times n}$, outputs $\mathbf{A} \in \mathbb{R}^{m\times n}$

1: Draw $\mathbf{T} \in \mathbb{R}^{k\times m}$ with i.i.d. $\mathcal{N}(0,1)$ entries
2: Run ICA on $\mathbf{TA}$ with $\epsilon = 1/\mathrm{poly}(m, d, k, \kappa(\mathbf{U}^*))$ to obtain $\widehat{\mathbf{TU}}$ with $\|\widehat{\mathbf{TU}} - \mathbf{TU}^*\mathbf{\Pi}\mathbf{D}\|_F \le \epsilon$, where $\mathbf{\Pi}$ is a permutation matrix and $\mathbf{D}$ diagonal.
3: Let $\overline{\mathbf{X}}, \overline{\mathbf{A}}$ be the first $\ell = \mathrm{poly}(d, m, k, \kappa(\mathbf{U}^*), \kappa(\mathbf{V}^*))$ columns of $\mathbf{X}, \mathbf{A}$, and $\tau = 1/\mathrm{poly}(\ell)$
4: $\hat{\mathbf{F}} \leftarrow (\widehat{\mathbf{TU}})^{-1}\mathbf{T}\overline{\mathbf{A}}$
5: **for** $i = 1, \dots, k$ **do**
6: $\quad S_i \leftarrow \{\, j \in [\ell] : \hat{F}_{ij} > \tau \,\}$
7: $\quad$ **for** $r = 1, \dots, k$ **do**
8: $\quad\quad$ Solve for $x \in \mathbb{R}^k$: $x^\top(\mathbf{T}\overline{\mathbf{A}})_{:,j} = 0$ for all $j \in [\ell] \setminus S_i$, $x_r = 1$
9: $\quad\quad$ **if** a solution exists **then** $\mathrm{w}_i \leftarrow x$ and **break**
10: $\quad$ **end for**
11: $\quad \mathbf{W}_{i,:} \leftarrow \mathrm{w}_i^\top \mathbf{T}\overline{\mathbf{A}}$
12: **end for**
13: Flip the signs of rows of $\mathbf{W}$ to make all entries non-negative
14: **for** $i = 1, \dots, k$ **do**
15: $\quad$ Solve $(\mathbf{W}_{i,:})_{S_i} = \mathrm{v}_i^\top \overline{\mathbf{X}}_{:,S_i}$ for $\mathrm{v}_i \in \mathbb{R}^d$
16: $\quad \mathrm{v}_i \leftarrow \frac{\mathrm{v}_i}{\|\mathrm{v}_i\|}$
17: **end for**
18: $\mathbf{V} \leftarrow [\mathrm{v}_1, \dots, \mathrm{v}_k]^\top$
19: Solve $\mathbf{A} = \mathbf{U}\,\sigma(\mathbf{VX})$ for $\mathbf{U}$ by Gaussian elimination
20: **return** $\mathbf{U}, \mathbf{V}$

Lines 7–11 recover the i-th row of $\sigma(\mathbf{V}^*\mathbf{X})$ exactly. $\mathrm{w}_i^\top\mathbf{TA}$ lies in the row space of $\sigma(\mathbf{V}^*\mathbf{X})$ and equals 0 outside S_i; conditioned on the correctness of which, $\mathbf{X}$ is tractable exactly. Bakshi et al. [2019] do not provide explicit constants for τ or ℓ.

2.5 The Gauss–Newton decomposition

For $f(z) = \|r(z)\|^2$, we denote the Gauss–Newton decomposition as such

$$\nabla^2 f = 2\mathbf{J}^\top\mathbf{J} + \mathbf{S},\ \mathbf{S} = 2\sum_i r_i \nabla^2 r_i.$$

Where, $\mathbf{J}^\top\mathbf{J} \succeq 0$; $\mathbf{S}$ vanishes when $r = 0$ [Botev, 2020]. Model-specific closed form is deferred to § 3.

2.6 Key Lemmas & Definitions

Lemma 1 (Weyl's inequality)**.** *[Horn and Johnson, 2012] Let* $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{p\times p}$ *be symmetric. Then*

$$\lambda_{\min}(\mathbf{A}+\mathbf{B}) \;\geq\; \lambda_{\min}(\mathbf{A}) + \lambda_{\min}(\mathbf{B}) \;\geq\; \lambda_{\min}(\mathbf{A}) - \|\mathbf{B}\|_2.$$

More generally, if $\mathbf{N} \in \mathbb{R}^{p\times r}$ *has orthonormal columns, then*

$$\lambda_{\min}(\mathbf{N}^\top(\mathbf{A}+\mathbf{B})\mathbf{N}) \;\geq\; \lambda_{\min}(\mathbf{N}^\top\mathbf{A}\mathbf{N}) - \|\mathbf{B}\|_2.$$

The second statement follows from the fact $\|\mathbf{N}^\top\mathbf{B}\mathbf{N}\|_2 \leq \|\mathbf{B}\|_2$. For our case, we define $\mathbf{N}$ to be an orthonormal basis of $T_{\text{sym}}^{\perp}$, with $\mathbf{N} \in \mathbf{R}^{p\times p-k}$ (see Proposition 1 for dimension justification).

Definition 1. *For a symmetric matrix* $\mathbf{M} \in \mathbb{R}^{p\times p}$*, define its smallest eigenvalue on the complement of the symmetry directions,*

$$\lambda_{\perp}(\mathbf{M}) \;\coloneqq\; \min_{\substack{z\perp T_{\text{sym}} \\ \|z\|=1}} z^\top\mathbf{M}z \;=\; \lambda_{\min}(\mathbf{N}^\top\mathbf{M}\mathbf{N}),$$

where $\mathbf{N} \in \mathbb{R}^{p\times(p-k)}$ *is any matrix whose columns form an orthonormal basis of* $T_{\text{sym}}^{\perp}$*.*

Lemma 2 (Matrix Chernoff, lower tail)**.** *Tropp [2012, Theorem 1.1] Let* $\mathbf{Y}_1, \ldots, \mathbf{Y}_n$ *be independent random symmetric* $r \times r$ *matrices satisfying*

$$\mathbf{Y}_j \succeq 0 \quad \textit{and} \quad \lambda_{\max}(\mathbf{Y}_j) \leq R \quad \textit{almost surely.}$$

Let $\mu_{\min} \coloneqq \lambda_{\min}\big(\sum_{j=1}^n \mathbb{E}\,\mathbf{Y}_j\big)$*. Then for every* $\varepsilon \in [0,1]$*,*

$$\mathbb{P}\Big\{\lambda_{\min}\Big(\sum_{j=1}^n \mathbf{Y}_j\Big) \leq (1-\varepsilon)\,\mu_{\min}\Big\} \;\leq\; r\cdot\left[\frac{e^{-\varepsilon}}{(1-\varepsilon)^{1-\varepsilon}}\right]^{\mu_{\min}/R}.$$

3 Main Results

3.1 Empirical Analysis of ICA-Driven Exact Recovery Algorithm

3.1.1 Experimental Setup

Data is generated according to the model of Section 2.2. The ground truth $\mathbf{U}^*$ has each entry i.i.d. $\mathcal{N}(0,1)$. To enforce the orthonormality of the rows of $\mathbf{V}^*$, we compute the QR decomposition of a $d \times d$ Gaussian matrix with each entry i.i.d. $\mathcal{N}(0,1)$, then fixing signs so that the diagonal of the triangular factor is positive, and take the first k rows of the orthogonal component as $\mathbf{V}^*$.

Proceeding to the implementation as outlined in Algorithm 1, ICA is applied to $\mathbf{TA}$, obtaining $\widehat{\mathbf{TU}^*}$ and setting $\hat{\mathbf{F}} = (\widehat{\mathbf{TU}^*})^{-1}\overline{\mathbf{TA}}$, where $\overline{\cdot}$ represents the restriction

to the ℓ columns. The ICA implementation used here is the default scikit-learn implementation of Hyvärinen [1999]'s FastICA, with parallelised algorithm. Since ICA can recover only up to sign and scale and our use case implies non-negativity through ReLU, we flip the signs of the rows of $\hat{\mathbf{F}}$, according to their mean. We then rescale each row such that the empirical standard deviation is $\sqrt{\frac{1}{2} - \frac{1}{2\pi}}$, matching that of a rectified Gaussian. Bakshi et al. [2019]'s algorithm only fixes $\hat{\mathbf{F}}$ up to a positive diagonal scaling, so a common τ is incoherent without standard scale. The sign fixing and rescaling enables interpretability of τ. With scale fixed and τ specified, sparsity patterns $\hat{S}_i$ are estimated.

Recovering the left null space of $\mathbf{T}\overline{\mathbf{A}}$ restricted to $[\ell]\backslash\hat{S}_i$ as specified in Lines 7–11 of Algorithm 1, is implemented through singular value decomposition (SVD); diverging from the paper's linear solve which fixes an arbitrary coordinate of w_i. At values near 0, the system is ill conditioned and when $\hat{S}_i$ is inaccurate, an exact $\mathrm{w}_i \in \mathrm{null}(\mathbf{M}^\top)$ does not exist, where $\mathbf{M} \coloneqq (\mathbf{T}\overline{\mathbf{A}})_{:,[\ell]\backslash\hat{S}_i}$. Support recovery has no guarantee at finite samples, so instead we take w_i as the left singular vector corresponding to the minimal singular value of the SVD of $\mathbf{M}$; equivalently, $\mathrm{w}_i \coloneqq \arg\min_{\|\mathrm{w_i}\|=1} \|\mathrm{w}_i^\top \mathbf{M}\|$, which coincides with linear solve at true $\hat{S}_i$.

Line 15 of Algorithm 1 uses Gaussian elimination to recover v_i, but on the support, the system is overdetermined (solving for $\dim(\mathrm{v}_i) = d$ unknowns with $|\hat{S}_i| \gg d$ constraints) and noisy due to support error. Consequently, least-squares is used in the implementation, solving $\hat{\mathrm{v}}_i = \arg\min_{\mathrm{v}\in\mathbb{R}^d} \|(\overline{\mathbf{X}}_{:,\hat{S}_i})^\top \mathrm{v} - \mathrm{w}_i^\top (\mathbf{T}\overline{\mathbf{A}})_{:,\hat{S}_i}\|^2$ using *all* equations, 'averaging' out the error, followed by enforcing $\|\mathrm{v}_i\| = 1$. Similarly, $\mathbf{U}^*$ is recovered as $\mathbf{A}\,\sigma(\mathbf{V}\mathbf{X})^\dagger$ by least squares (Line 13 Algorithm 2)

Our implementation (below) therefore follows a slightly modified version of Algorithm 1, albeit mathematically identical given favourable conditions.

Algorithm 2 Finite-Sample Implementation of Algorithm 1

Require: Inputs $\mathbf{X}$, outputs $\mathbf{A}$, number of samples ℓ, threshold τ

Draw $\mathbf{T} \in \mathbb{R}^{k \times m}$ with i.i.d. $\mathcal{N}(0,1)$ entries

Run FastICA (default scikit-learn parameterisation) on $\mathbf{TA}$ to obtain $\widehat{\mathbf{TU}}$, and set $\hat{\mathbf{F}} \leftarrow (\widehat{\mathbf{TU}})^{-1}\mathbf{TA}$

for $i = 1, \dots, k$ **do**

  $\hat{\mathbf{F}}_{i,:} \leftarrow \operatorname{sign}\big(\operatorname{mean}(\hat{\mathbf{F}}_{i,:})\big)\, \hat{\mathbf{F}}_{i,:}$, then rescale so that $\operatorname{std}(\hat{\mathbf{F}}_{i,:}) = \sqrt{\frac{1}{2} - \frac{1}{2\pi}}$

end for

Let $\overline{\mathbf{X}}, \overline{\mathbf{A}}, \overline{\mathbf{F}}$ be the first ℓ columns of $\mathbf{X}, \mathbf{A}, \hat{\mathbf{F}}$

for $i = 1, \dots, k$ **do**

  $\hat{S}_i \leftarrow \{\, j \in [\ell] : \overline{F}_{ij} > \tau \,\}$

  $\mathrm{w}_i \leftarrow \arg\min_{\|\mathrm{w}\|=1} \big\|\mathrm{w}^\top (\mathbf{T}\overline{\mathbf{A}})_{:,[\ell]\setminus\hat{S}_i}\big\|$ ▷ smallest singular vector

  $\mathrm{r}_i \leftarrow \mathrm{w}_i^\top \mathbf{T}\overline{\mathbf{A}}$, then $\mathrm{r}_i \leftarrow \operatorname{sign}(\operatorname{mean}(\mathrm{r}_i))\, \mathrm{r}_i$

  $\hat{\mathrm{v}}_i \leftarrow \arg\min_{\mathrm{v}} \big\|\overline{\mathbf{X}}_{:,\hat{S}_i}^\top \mathrm{v} - (\mathrm{r}_i)_{\hat{S}_i}\big\|^2, \quad \mathrm{v}_i \leftarrow \hat{\mathrm{v}}_i / \|\hat{\mathrm{v}}_i\|$ ▷ least squares

end for

$\mathbf{V} \leftarrow [\mathrm{v}_1, \dots, \mathrm{v}_k]^\top, \quad \mathbf{U} \leftarrow \arg\min_{\mathbf{U}} \|\mathbf{U}\,\sigma(\mathbf{VX}) - \mathbf{A}\|_F = \mathbf{A}\,\sigma(\mathbf{VX})^\dagger$

return $\mathbf{U}, \mathbf{V}$

Thresholding parameters Throughout, we fix $\ell = 500$. Since Bakshi et al. [2019] specify τ only as $1/\mathrm{poly}(\ell)$, we either sweep τ (Section 3.1.3) or give Algorithm 2 its best case, where τ is selected by minimising support error against ground truth sparsity. We refer to the latter as the *oracle-tuned* τ. This is used only to bound the algorithm's error, as this is infeasible in practice.

Support error Let $S_i^* = \{j \in [\ell] : (\mathbf{V}^*\overline{\mathbf{X}})_{ij} > 0\}$ denote the true support of hidden unit i. We record false positive and false negative rates

$$\mathrm{FP} = \frac{1}{k\ell} \sum_{i=1}^{k} |\hat{S}_i \setminus S_i^*|, \qquad \mathrm{FN} = \frac{1}{k\ell} \sum_{i=1}^{k} |S_i^* \setminus \hat{S}_i|,$$

and total support error rate FP + FN.

Gradient descent We minimise objective function $f(\mathbf{U}, \mathbf{V})$ by full-batch Adam with learning rate 0.01 and cosine annealing. Unless specified otherwise, $\mathbf{U}$ is initialised with i.i.d. $\mathcal{N}(0,1)$ entries and the rows of $\mathbf{V}$ are i.i.d. Gaussian, normalised to unit norm. We return the iterate with the lowest loss, and report loss per entry $f/(mn)$. We say an estimate reaches machine precision when its relative errors $\varepsilon_{(\hat{\mathbf{U}},\hat{\mathbf{V}})}$ are of order 10^{-12}, consequence of the arithmetic bottleneck of the alignment stage.

Configurations Table 1 lists the configurations used in each experiment.

Table 1: Experimental configurations.

Experiment	(m, k, d)	n	p	q
Recovery error, threshold analysis (§3.1.2–3.1.4)	$(30, 15, 25)$	20,000	0	0
Robustness (§3.1.5)	$(30, 15, 25)$	20,000; 100,000	0; 0.1	0; 0.1
ICA initialisation (§3.2)	$(30, 15, 25)$	20,000	0	0
Curvature (§3.3.3)	$(8, 4, 6)$	2,000	0	0

3.1.2 Recovery Error in the Finite-Sample Regime

Table 2 compares Algorithm 2 with gradient descent as described in §3.1.1 at $(m, k, d) = (30, 15, 25)$ and $n = 20{,}000$. Supplied with the true support, Algorithm 2 recovers both weight matrices to machine precision, confirming that the linear-algebraic steps are exact. End to end implementation of Algorithm 2, even with the oracle-tuned threshold $\tau = 0.212$, 8.8% of the support is misclassified, almost entirely as false negatives, and the recovery error rises to $\varepsilon = (0.055, 0.075)$. On the other hand, gradient descent from a random initialisation reaches machine precision.

Table 2: Recovery error at $(m, k, d) = (30, 15, 25)$, $n = 20{,}000$, noiseless.

Method	ε_U	ε_V	Support error
Algorithm 2, true support	1.0×10^{-12}	1.0×10^{-12}	0
Algorithm 2, oracle-tuned τ	0.055	0.075	8.8%
Gradient descent, random init.	1.0×10^{-12}	1.1×10^{-12}	–

3.1.3 Thresholding Bottleneck

Given true support, the algorithm is exact, so its error lies in the thresholding step. To separate the two sources of error, Table 3 varies τ , comparing support error from thresholding the true activations $\sigma(\mathbf{V}^*\overline{\mathbf{X}})$ versus thresholding the ICA estimated $\hat{\mathbf{F}}$. Note that thresholding true activations can only give false negatives, at rate $\mathbb{P}(\{0 \le \sigma(\mathbf{V}^*_{i,:}\mathrm{x}_j) \le \tau\}) = \Phi(\tau) - \frac{1}{2}$, where $\Phi(\cdot)$ represents the standard Gaussian cumulative distribution function. At small τ, the gap between the two columns is dominated by ICA noise, pushing true zeros above the threshold (false positives). As τ grows the gap closes, but small genuine activations are discarded even from the true activations. At finite n it is such the case that no choice of τ can give an accurate estimate of sparsity and at best one can only approach the bottleneck of ICA induced error.

Table 3: Support error rate (FP + FN) against τ.

τ	True activations	ICA estimate	Gap
0.0001	0.0001	0.2644	0.2643
0.01	0.0052	0.2124	0.2072
0.02	0.0104	0.1687	0.1583
0.05	0.0223	0.0795	0.0572
0.10	0.0444	0.0516	0.0072
0.212	0.0895	0.0912	0.0017

3.1.4 Asymmetric Cost of Support Errors and Optimising τ

The effect of each error type on recovery is not equal. A false positive admits an unbounded incorrect value into the null-space step for that unit, whereas a false negative only discards a value smaller than τ, yet imposes a false constraint (see Line 9 of Algorithm 2). To quantify this, we inject a controlled number of errors of each type into the true support and run the remainder (Lines 7–14) of Algorithm 2 (Table 4). At five injected errors per row, false positives are roughly 75 times more costly; the two converge only once the support is heavily corrupted.

Table 4: $\hat{\mathbf{V}}$ recovery error after injecting errors into the oracle support with $(m, k, d) = (30, 15, 25)$, $n = 20{,}000$

Injected errors per row (%)	1	2	4	8	16
False negatives	0.00065	0.00155	0.00406	0.01604	0.14488
False positives	0.04914	0.05892	0.08284	0.12063	0.14350

Some rough heuristics for τ The rescaled ICA estimate can be modelled as $\hat{F}_{ij} = \sigma(\mathrm{v}_i^{*\top}\mathrm{x}_j) + \varepsilon_{ij}$ with $\varepsilon_{ij} \sim \mathcal{N}(0, \hat{\sigma}^2)$, where $\hat{\sigma}^2$ can be estimated from the negative entries of $\hat{\mathbf{F}}$ as

$$\hat{\sigma}^2 = |\Omega|^{-1} \sum_{(i,j)\in\Omega} \hat{F}_{ij}^2 \quad \text{with} \quad \Omega = \{(i,j) : j \le \ell,\ \hat{F}_{ij} < 0\}$$

(the case in Table 4 yields $\hat{\sigma} \approx 0.04$).

An entry is declared active if $\hat{F}_{ij} > \tau$. With Φ the standard normal CDF, the per-entry error rates are simply tail probabilities

$$\mathrm{FP}(\tau) = \tfrac{1}{2}(1 - \Phi(\tau/\hat{\sigma})), \qquad \mathrm{FN}(\tau) = \int_0^\infty \phi(u)\, \Phi\!\left(\frac{\tau - u}{\hat{\sigma}}\right) du,$$

the latter being the probability that a true activation $u > 0$ is pushed below τ by noise: for observed entry

$$u + \varepsilon,\ \varepsilon \sim \mathcal{N}(0, \hat{\sigma}^2) \implies \text{inactive w.p. } \mathbb{P}(\{u + \varepsilon < \tau\}) = \Phi\!\left(\frac{\tau - u}{\hat{\sigma}}\right).$$

As $\hat{\sigma} \to 0$, $\mathrm{FN}(\tau) \to \Phi(\tau) - \frac{1}{2}$, and we approximate it as such henceforth.

Under configuration $(m, k, d) = (30, 15, 25)$, $n = 20{,}000$, $\ell = 500$, observe the following results.

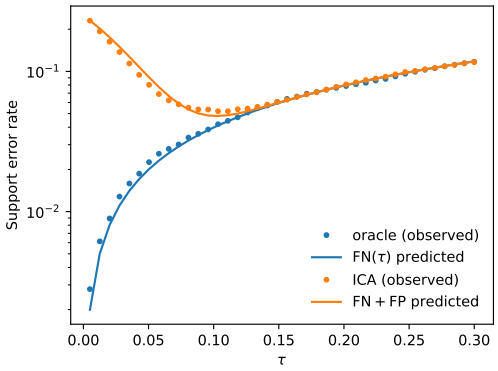


Figure 1: Support error rate against τ. Points are observed, lines are the predictions $\mathrm{FN}(\tau)$ and $\mathrm{FN}(\tau) + \mathrm{FP}(\tau)$, with $\hat{\sigma}$ estimated from negative entries of $\hat{\mathbf{F}}$. Support error minimised near $\tau = 0.10$.

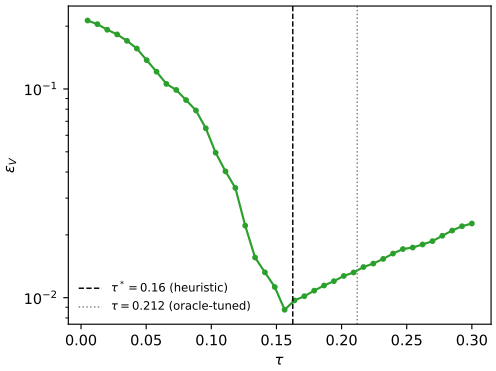


Figure 2: Recovery error ε_V against τ. False positives cost more than false negatives, giving a steep rise to left of τ^* and gradually to the right. See explanation below for τ^*. Oracle tuned τ admits excess false negatives.

As seen in Table 4, false negatives near zero are cheap, wheras false positives are costly. We hypothesise that optimal τ^* should be the smallest threshold at which all false positives vanish. Note the minimum in Figure 2 coincides with the point of convergence of $\mathrm{FN}(\tau)$ and $\mathrm{FN}(\tau) + \mathrm{FP}(\tau)$ in Figure 1. In expectation, requiring less than one false positive among $k\ell$ entries gives

$$k\ell \cdot \tfrac{1}{2}\big(1 - \Phi(\tau/\hat{\sigma})\big) \leq 1 \quad \Longrightarrow \quad \tau^* = \hat{\sigma}\, \Phi^{-1}\!\left(1 - \frac{2}{k\ell}\right),$$

which coincides with the minimiser of ε_V in Figure 2. Using our empirical estimate for $\hat{\sigma} \approx 0.04$, we have that $\tau^* = 0.16$ compared to the empirically determined optimal 0.15445. It follows that tuning τ to maximise sparsity correctness is misguided, as excess false negatives are permissible so long as false positives are avoided.

Possibilities for Further Optimisation Once τ is large enough to limit false positives, we can identify active units with a certain level of confidence. These active units are the ones used in Line 11 of Algorithm 2 to recover rows of $\mathbf{V}$ $\big(\arg\min_{\mathrm{v}} \big\| \overline{\mathbf{X}}_{:,\hat{S}_i}^{\top} \mathrm{v} - (\mathrm{r}_i)_{\hat{S}_i} \big\|^2\big)$ by restricting on $\hat{S}_i$. τ which maximises confidence in $\hat{S}_i$ minimises the error propagated through this step. Analogously, Line 9 imposes constraints based on estimated zeros $\big(\arg\min_{\|\mathrm{w}\|=1} \big\| \mathrm{w}^{\top} (\mathbf{T}\overline{\mathbf{A}})_{:,[\ell]\setminus\hat{S}_i} \big\| \big)$ through $[\ell] \setminus \hat{S}_i$. There remains the possibility of imposing a $\{j \in [\ell] : \hat{F}_{ij} < \theta\}$ mask to ascertain the zeros of high confidence as opposed to taking the complement of $\hat{S}_i$ which retains ambiguous activations. So long as the number of linearly independent active and inactive units (hence constraints) remaining after such thresholding provides

sufficient parameterisation for the respective linear steps, it is not unreasonable to hypothesise that Algorithm 2 could be made significantly more accurate. We leave the implementation of such changes to future work.

3.1.5 Robustness to Additive Noise and Sample Contamination

Table 5 shows recovery error under additive noise and contamination at $n = 100{,}000$, see §2.2 for model specification. Algorithm 2 degrades sharply as 10% contamination impacts its error at a factor of ≈ 85, almost wholly through false positives in the recovered support. Additive noise alone raises it to 0.34. Observe that gradient descent decays gracefully, at its worst only degrading to 0.0335 with corruption. We find that gradient descent also converges to the similar final error regardless of initialisation; under 10% contamination at $n = 20{,}000$, random and Algorithm 2 initialisations both give ≈ 0.043 with the same seed (not shown in Table 5).

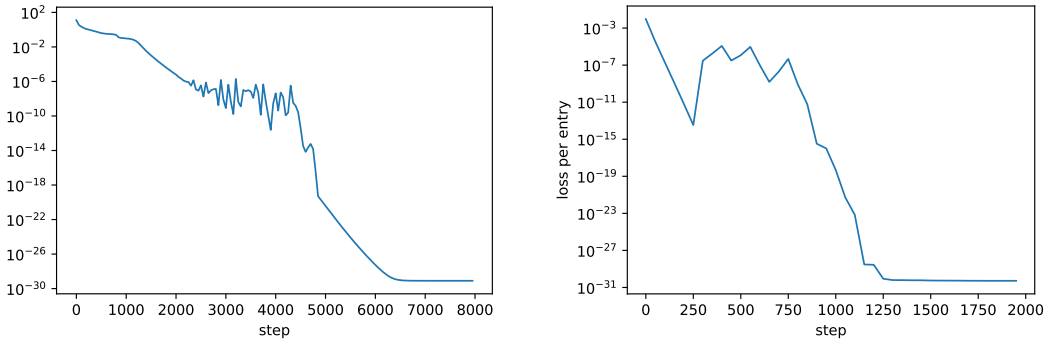
Table 5: $\hat{\mathbf{V}}$ recovery error under noise and contamination, $n = 100{,}000$.

Data	Algorithm 2	Gradient descent
Clean ($p = q = 0$)	0.0074	0.0000
Noise ($p = 0.1$)	0.3439	0.0005
Contamination ($q = 0.1$)	0.6289	0.0335

3.2 ICA Initialisation with GD Refinement

Given robustness of gradient descent and observed invariance with respect to distinct initialisations, it is reasonable to doubt the relevance of Algorithm 2. However, in spite of its seemingly lacklustre performance, it serves as a useful initialiser for further gradient descent based refinement. We have observed a non-trivial increase in the speed of GD convergence up to ≈ 4 times quicker compared to random initialisation, demonstrated in the following empirical results and explanations.

3.2.1 Convergence Speed and Final Error



(a) Random initialisation

(b) Initialisation from Algorithm 2

Figure 3: Training loss per entry of gradient descent, $(m, k, d) = (30, 15, 25)$, $n =$ 20,000. From ICA-based initialisation, machine precision is reached in roughly a quarter of the steps.

Table 6 shows the number of steps for each initialisation to reach close to machine precision and final recovery error for $\hat{\mathbf{V}}$. Reporting the best iterate inflates the step count as the incremental improvement over a couple thousand more steps is minimal, so instead we report an eyeballed figure of where the per-entry squared loss begins to plateau at $\approx 10^{-31}$, approaching machine precision for float64.

Table 6: Steps for gradient descent to reach machine precision.

Initialisation	Steps to machine precision	Final $\varepsilon_{\hat{\mathbf{V}}}$
Random	$\approx$ 5,300	1.06×10^{-12}
Spectral	$\approx$ 5,200	1.05×10^{-12}
Algorithm 2	$\approx$ 1,250	9.67×10^{-13}

3.2.2 Why ICA helps: Orientation versus Subspace

To understand what the ICA initialisation recovers, we investigate two angles. The *row-space angle* is the largest principal angle between the row spaces of $\hat{\mathbf{V}}$ and $\mathbf{V}^*$. The *per-unit angle* is $\theta_i = \arccos |\langle \hat{\mathrm{v}}_i, \mathrm{v}^*_{\pi(i)} \rangle|$ for normalised rows, after Hungarian algorithm matching on $|\cos(\cdot)|$ ($\pi(i)$ representing the resulting permutation). The row-space angle is rotationally invariant within the span, which is precisely the degree of freedom GD must resolve, i.e. find its basis (rows of $\mathbf{V}^*$). On the other hand, the per-unit angle is invariant only to the inherent ambiguity of the optimisation problem: permutation and rescaling. Applying an orthonormal $\mathbf{Q}$ to $\mathbf{V}^*$ cannot change its span, but reorients the k dimensional sphere it represents. Intuitively, one can think

of Algorithm 2 as a means of narrowing that rotational ambiguity instead of as a self contained end-to-end recovery algorithm.

Table 7: Initialisation quality before gradient descent, $(m, k, d) = (30, 15, 25)$, $n =$ 20,000, mean over 5 seeds. (Angles in radians)

Initialisation	Row-space	Per-unit (mean)	ε_V
ICA	0.010	0.013	0.015
Spectral	0.081	1.042	1.099
Random	1.531	1.224	1.387

Spectral initialisation recovers row space of $\mathbf{V}^*$ within 0.08 rad, as SVD can identify the span but not its basis. As a result, per unit it is no better than random. ICA recovers each hidden unit's direction to 0.013 rad on average. The remaining rotational ambiguity is what gradient descent narrows (as opposed to finding a subspace), kind of 'tilting' the k dimensional unit sphere determined by $\mathbf{V}^*$ around the origin.

For a different, but less hand-wavy explanation, see §3.3.3.

3.3 Second-order Geometry of the Loss

3.3.1 Hessian and its Symmetry

Quadratic form Consider the function $g(t) = f(\mathbf{U} + t\,\Delta\mathbf{U}, \mathbf{V} + t\,\Delta\mathbf{V})$, for some arbitrary perturbation directions $\Delta\mathbf{U}$, $\Delta\mathbf{V}$. If $g''(0) \geq 0\ \forall\, \Delta\mathbf{U}, \Delta\mathbf{V}$, then it must be the case that $\mathbf{H} = \nabla^2 f \succeq 0$. We first try to formulate an expression for $g(t)$ as such:

$$g(t) = \left\|(\mathbf{U} + t \cdot \Delta\mathbf{U})\sigma((\mathbf{V} + t \cdot \Delta\mathbf{V})\mathbf{X}) - \mathbf{A}\right\|_F^2$$

Using a Taylor's expansion argument as described in Appendix A.1, we derive the quadratic form as such:

$$\begin{aligned} g''(0) &= 4\langle \mathbf{P}_0, \mathbf{P}_2 \rangle + 2\|\mathbf{P}_1\|_F^2 \\ &= 4\big\langle \mathbf{U}\,\sigma(\mathbf{V}\mathbf{X}) - A, \Delta\mathbf{U}\sigma'(\mathbf{V}\mathbf{X}) \odot (\Delta\mathbf{V}\mathbf{X}) \big\rangle \\ &\qquad + 2\left\|\mathbf{U}\,\sigma'(\mathbf{V}\mathbf{X}) \odot (\Delta\mathbf{V}\mathbf{X}) + \Delta\mathbf{U}\,\sigma(\mathbf{V}\mathbf{X})\right\|_F^2 \end{aligned}$$

Note that the second term is ≥ 0 by definition, so all of the 'concavity' of loss lies in the term on the left side, which is proportional to the residual $\mathbf{U}\,\sigma(\mathbf{V}\mathbf{X}) - \mathbf{A}$.

Closed Form Hessian in terms of Kronecker Product We derive the closed form of the Hessian of $f(\mathbf{U}, \mathbf{V}) = \|\mathbf{U}\,\sigma(\mathbf{V}\mathbf{X}) - \mathbf{A}\|_F^2$ in Appendix A.2 as such:

$$\mathbf{H} = \nabla^2 f = \begin{bmatrix} \mathbf{H}_{\mathbf{UU}} & \mathbf{H}_{\mathbf{UV}} \\ \mathbf{H}_{\mathbf{UV}}^\top & \mathbf{H}_{\mathbf{VV}} \end{bmatrix}$$

where

$$\begin{aligned}
\mathbf{H_{UU}} &= 2(\mathbf{F}\mathbf{F}^\top \otimes \mathbf{I}_m) \\
\mathbf{H_{UV}} &= 2\left[(\mathbf{F} \otimes \mathbf{U}) + (\mathbf{I}_k \otimes \mathbf{R})\mathbf{K}_{k,n}\right]\mathbf{D}(\mathbf{X}^\top \otimes \mathbf{I}_k) \\
\mathbf{H}_{\mathbf{UV}}^\top &= 2(\mathbf{X} \otimes \mathbf{I}_k)\mathbf{D}\left[(\mathbf{F}^\top \otimes \mathbf{U}^\top) + \mathbf{K}_{n,k}(\mathbf{I}_k \otimes \mathbf{R}^\top)\right] \\
\mathbf{H_{VV}} &= 2(\mathbf{X} \otimes \mathbf{I}_k)\mathbf{D}(\mathbf{I}_n \otimes \mathbf{U}^\top\mathbf{U})\mathbf{D}(\mathbf{X}^\top \otimes \mathbf{I}_k)
\end{aligned}$$

With $\mathbf{F} = \sigma(\mathbf{VX}), \mathbf{R} = \mathbf{UF} - \mathbf{A}, \mathbf{D} = \operatorname{diag}(\operatorname{vec}(\sigma'(\mathbf{VX})))$, as defined in §2.1.

The expression been verified to machine precision against autodiff.

Gauss–Newton Decomposition Writing $\mathbf{z} = [\operatorname{vec}(\mathbf{U}); \operatorname{vec}(\mathbf{V})] \in \mathbb{R}^{mk+kd}$, observe that $f = \|\operatorname{vec}(\mathbf{R})\|^2$. Let $\mathbf{J}$ denote the Jacobian of the residual,

$$\mathbf{J} = \frac{\partial \operatorname{vec}(\mathbf{R})}{\partial \mathbf{z}^\top} = \begin{bmatrix}\mathbf{J_U} & \mathbf{J_V}\end{bmatrix} \in \mathbb{R}^{mn\times(mk+kd)},$$

$$\begin{aligned}
\mathbf{J_U} &= \mathbf{F}^\top \otimes \mathbf{I}_m \\
\mathbf{J_V} &= (\mathbf{I}_n \otimes \mathbf{U})\,\mathbf{D}\,(\mathbf{X}^\top \otimes \mathbf{I}_k)
\end{aligned}$$

Then the gradient and Hessian admit the Gauss–Newton decomposition

$$\begin{aligned}
\nabla f &= 2\mathbf{J}^\top \operatorname{vec}(\mathbf{R}) \\
\nabla^2 f &= \underbrace{2\mathbf{J}^\top\mathbf{J}}_{\succeq 0} + \mathbf{S}
\end{aligned}$$

where $\mathbf{S}$ collects the second-order terms arising from differentiating $\mathbf{J}$ itself. These vanish on the diagonal blocks. Can also think of this as writing the Hessian as a multiplication of matrix and its transpose, and $\mathbf{S}$ being the remainder of that. Hence

$$\mathbf{S} = \begin{bmatrix}\mathbf{0} & \mathbf{S_{UV}} \\ \mathbf{S}_{\mathbf{UV}}^\top & \mathbf{0}\end{bmatrix}, \qquad \mathbf{S_{UV}} = 2(\mathbf{I}_k \otimes \mathbf{R})\mathbf{K}_{k,n}\,\mathbf{D}\,(\mathbf{X}^\top \otimes \mathbf{I}_k)$$

Knowing the structure of $\mathbf{S}$, we can now make some claims regarding its norm and how it relates to the Hessian's curvature. The dependence of $\mathbf{S}$ on $\mathbf{R}$ means that at ground truth, $\nabla^2 f = 2\mathbf{J}^\top\mathbf{J}$.

Define the *population* Gauss–Newton matrix as

$$\Sigma^*(\mathbf{U}, \mathbf{V}) \coloneqq \mathbb{E}_{\mathrm{x}\sim\mathcal{N}(\mathbf{0},\mathbf{I}_d)}[\mathbf{J}(\mathrm{x})^\top\mathbf{J}(\mathrm{x})],$$

noting $\mathbb{E}[\nabla^2 f] = 2\Sigma^*$.

Similarly, denote the *empirical* Gauss–Newton matrix as

$$\hat{\mathbf{G}}(\mathbf{U}, \mathbf{V}) \coloneqq \frac{1}{n}\mathbf{J}^\top\mathbf{J} = \frac{1}{n}\sum_{j=1}^{n}\mathbf{J}(\mathrm{x}_j)^\top\mathbf{J}(\mathrm{x}_j)$$

We proceed to define the symmetry tangent space as:

$$T_{\text{sym}}(\mathbf{U}, \mathbf{V}) \coloneqq \{\mathrm{z}_b = (\mathbf{U}\,\mathrm{diag}(\mathrm{b}),\ -\mathrm{diag}(\mathrm{b})\,\mathbf{V}) : \mathrm{b} \in \mathbb{R}^k\}.$$

Along any direction in T_{sym}, the loss is unchanged and the Jacobian is trivially zero and therefore contained in the kernel of Σ^*.

Lemma 3 (Symmetry directions are always null)**.** *For every* $(\mathbf{U}, \mathbf{V})$*, every* x *and every* b*, we have* $\mathbf{J}(\mathrm{x})\,\mathrm{z}_b = 0$*. Hence* $T_{\text{sym}} \subseteq \ker \mathbf{J}^\top \mathbf{J}$ *and* $T_{\text{sym}} \subseteq \ker \Sigma^*$*.*

Proof.

$$\begin{aligned}\mathbf{J}(\mathrm{x})\,\mathrm{z}_b &= \mathbf{U}\,\mathrm{diag}(\mathrm{b})\,\sigma(\mathbf{V}\mathrm{x}) - \mathbf{U}(\sigma'(\mathbf{V}\mathrm{x}) \odot \mathrm{diag}(\mathrm{b})\mathbf{V}\mathrm{x}) \\ &= \mathbf{U}\,\mathrm{diag}(\mathrm{b})(\sigma(\mathbf{V}\mathrm{x}) - \sigma'(\mathbf{V}\mathrm{x}) \odot \mathbf{V}\mathrm{x}) = 0,\ \text{since } \sigma(t) = \mathbf{1}[t > 0]\,t.\end{aligned}$$

□

3.3.2 Curvature at Ground Truth

At $(\mathbf{U}^*, \mathbf{V}^*)$ the residual vanishes, so $\nabla^2 f = 2\mathbf{J}^\top \mathbf{J}$ and the curvature is governed entirely by the Gauss–Newton term. By Lemma 3, $\mathbf{J}^\top \mathbf{J}$ singular along T_{sym} so we consider the relative quantity $\lambda_\perp(\cdot)$ (Definition 1) representing the minimum eigenvalue projected away from T_{sym}. The following proposition claims that those very symmetry direction are *exactly* the flat directions of the population Gauss–Newton matrix.

Proposition 1 (The population Gauss-Newton kernel is exactly the symmetry tangent space)**.** *Suppose* x *has a density that is positive on* $\mathbb{R}^d$ *(e.g.* $\mathrm{x} \sim \mathcal{N}(0, I_d)$*), U has full column rank (*$m \geq k$*), and V has full row rank (*$k \leq d$*). Then* $\ker \Sigma^*(\mathbf{U}, \mathbf{V}) = T_{\text{sym}}(\mathbf{U}, \mathbf{V})$*.*

Proof sketch. The inclusion $\supseteq$ is Lemma 3. For the other direction, we show that for any perturbation z in parameter space such that $\mathrm{z} \in \ker \Sigma^*$ forces $\mathbf{J}(\mathrm{x})\mathrm{z} = 0$ for almost every x. Evaluating on the orthant where all units active forces both perturbations into the column space of $\mathbf{U}$ and row space of $\mathbf{V}$ and can therefore be represented as $\mathbf{UB}$ and $\mathbf{AV}$. By comparing orthants that differ in a single active unit forces $\mathbf{B} = \mathrm{diag}(\mathrm{b})$, $\mathbf{A} = -\mathrm{diag}(\mathrm{b})$ so $\mathrm{z} = \mathrm{z}_b \in T_{\text{sym}}$. See Appendix B.2 for full proof. □

The ReLU removes all but the diagonal rescalings, so the orientation of each hidden unit becomes identifiable; this is precisely the degree of freedom that the ICA initialisation resolves and the spectral initialisation does not (§3.2.2).

The ground truth is therefore an equivalence class of strict local minima having projected out kernel T_{sym}. The next result helps connect this to the *empirical* Hessian at ground truth.

Recall that $\hat{\mathbf{G}} = \frac{1}{n}\mathbf{J}^\top\mathbf{J}$ is the empirical Gauss–Newton matrix and define $\mu^* \coloneqq \lambda_\perp(\Sigma^*) > 0$ as the minimum off-symmetry eigenvalue of its population counterpart (Proposition 1). Let $p = mk + kd$ denote the number of parameters.

The empirical matrix is an average of n independent per-sample terms $\mathbf{J}(\mathrm{x}_j)^\top\mathbf{J}(\mathrm{x}_j)$, each with mean Σ^*, so its curvature should concentrate around μ^* given that n is sufficiently large. The main obstacle is that each term grows with $\|\mathrm{x}_j\|^2$ hence unbounded. We handle this by truncating the inputs at R_x, under which each term is bounded by

$$R \coloneqq (\|\mathbf{U}^*\|_2^2 + \|\mathbf{V}^*\|_2^2)\, R_\mathrm{x}^2.$$

We want to be able to bound the minimum eigenvalue of the off symmetry empirical Gauss–Newton $\lambda_\perp(\hat{\mathbf{G}})$, in terms of that of the population, which we shall define as

$$\mu^* = \lambda_\perp(\Sigma^*) = \lambda_{\min}\big(\mathbf{N}^\top\Sigma^*\mathbf{N}\big),$$

where $\mathbf{N}$ refers to an orthonormal basis of $T_{\mathrm{sym}}^\perp$ as described in §1 .

Theorem 1 (Empirical curvature at ground truth)**.** *Suppose the conditions of Proposition 1 hold at* $(\mathbf{U}^*, \mathbf{V}^*)$*, and let* $\delta \in (0, 1)$*. If*

$$n \;\geq\; \frac{24R}{\mu^*}\log\frac{2(p-k)}{\delta},$$

then with probability at least $1 - \delta$*,*

$$\lambda_\perp(\hat{\mathbf{G}}) \;\geq\; \frac{\mu^*}{2}.$$

Here R_x *is chosen, as in Appendix B.3, so that the truncation changes* μ^* *by at most* $\mu^*/4$ *and no input exceeds* R_x *with probability at least* $1 - \delta/2$*; it grows like* $\sqrt{d}$ *plus a logarithmic term.*

Proof sketch. Project each per-sample term onto $T_{\mathrm{sym}}^\perp$ and considering truncation event $\|\mathrm{x}_j\| \leq R_\mathrm{x}\ \forall\, j$ so that each term is bounded by R. This ensures the *maximum* eigenvalue of that which we are relating to μ^* is bounded, as is required by Lemma 2. Truncation induces bias (controlled via Lemma 5) and failure probability of truncation event controlled via a union bound. Once controlled, conclusion follows via standard application of matrix Chernoff bound. See Appendix B.3 for full proof. □

3.3.3 Curvature away from Ground Truth

Away from the ground truth the residual $\mathbf{R} = \mathbf{U}\,\sigma(\mathbf{V}\mathbf{X}) - \mathbf{A}$ is nonzero, and the term $\mathbf{S}$ in the decomposition $\nabla^2 f = 2\mathbf{J}^\top\mathbf{J} + \mathbf{S}$ can contribute negative curvature. The following bound controls $\mathbf{S}$ by the size of the residual alone, which leads to a sufficient condition for positive curvature that can be checked at any iterate.

Lemma 4 (Residual bound on $\mathbf{S}$)**.** *For any* $(\mathbf{U}, \mathbf{V})$*,* $\|\mathbf{S}\|_2 \leq 2\,\|\mathbf{R}\|_F\,\|\mathbf{X}\|_2$.

Proof sketch. Bound the quadratic form $\mathrm{z}^\top \mathbf{S}\mathrm{z} = 4\langle \mathbf{R},\ \Delta\mathbf{U}\,[\mathbf{1}(\mathbf{V}\mathbf{X} > 0) \odot \Delta\mathbf{V}\mathbf{X}]\rangle$ by Cauchy–Schwarz. Activation mask can only decrease the Frobenius norm, and then apply $\|\Delta\mathbf{U}\|_F\|\Delta\mathbf{V}\|_F \le \frac{1}{2}\|\mathrm{z}\|^2$. See Appendix B.4. □

Corollary 1 (Curvature certificate)**.** *If* $\|\mathbf{R}\|_F < \frac{\lambda_\perp(\mathbf{J}^\top\mathbf{J})}{\|\mathbf{X}\|_2}$*, then* $\lambda_\perp(\nabla^2 f) > 0$.

Proof. By Lemma 1 and Lemma 4, $\lambda_\perp(\nabla^2 f) \ge 2\,\lambda_\perp\left(\mathbf{J}^\top\mathbf{J}\right) - 2\,\|\mathbf{R}\|_F\|\mathbf{X}\|_2 > 0$. □

We can compute both sides of the relationship at any iterate, so Corollary 1 certifies positive curvature off of the symmetry directions along a training run (or initialisation). For Gaussian inputs, $\|\mathbf{X}\|_2 \le \sqrt{n} + \sqrt{d} + t$ with probability at least $1 - 2e^{-t^2/2}$ [Vershynin, 2018, Corollary 7.3.3], so in normalised form it suffices that

$$\frac{\|\mathbf{R}\|_F}{\sqrt{n}} \;<\; \frac{\lambda_\perp\left(\frac{1}{n}\mathbf{J}^\top\mathbf{J}\right)}{1 + \frac{\sqrt{d}+t}{\sqrt{n}}},$$

where, by Theorem 1, the right-hand side is at least about $\mu^*/2$ near the ground truth for large enough n. We find that the certificate is in fact conservative, demonstrated in the following section.

3.3.4 Numerical Validation

The closed-form Hessian (§3.3) is dense of size $p \times p$ with Kronecker factors of size $mn \times p$, so due to computational limitations, we validate at $(m, k, d) = (8, 4, 6)$, $n =$ 2,000. Perturbing the ground truth along a random direction, $(\mathbf{U}^* + r\,\Delta\mathbf{U},\ \mathbf{V}^* + r\,\Delta\mathbf{V})$, and at each r we compute the residual, the certificate threshold $\lambda_\perp(\mathbf{J}^\top\mathbf{J})/\|\mathbf{X}\|_2$, and the true $\lambda_\perp(\nabla^2 f)$.

Table 8: Curvature certificate along a random perturbation of the ground truth, $(m, k, d) = (8, 4, 6)$, $n =$ 2,000. The certificate holds when the normalised residual (LHS) is below the threshold (RHS).

r	LHS: $\frac{\|\mathbf{R}\|_F}{\sqrt{n}}$	RHS: $\frac{\lambda_\perp(\mathbf{J}^\top\mathbf{J})}{\sqrt{n}\,\|\mathbf{X}\|_2}$	Holds	Certified bound	True $\frac{\lambda_\perp(\nabla^2 f)}{n}$
0	0.000	0.168	✓	0.347	0.347
0.01	0.048	0.169	✓	0.250	0.349
0.03	0.144	0.172	✓	0.057	0.350
0.05	0.240	0.173	×	−0.137	0.344
0.1	0.478	0.170	×	−0.633	0.313
0.2	0.946	0.164	×	−1.611	0.201
0.3	1.410	0.155	×	−2.584	0.015
0.4	1.875	0.152	×	−3.550	−0.206

The certificate holds up to $\frac{\|\mathbf{R}\|_F}{\sqrt{n}} \approx 0.14$ ($r = 0.03$), whereas $\lambda_\perp(\nabla^2 f)$ remains positive up to $\frac{\|\mathbf{R}\|_F}{\sqrt{n}} \approx 1.4$ ($r \approx 0.3$); the certificate is therefore roughly $10\times$ conservative. This discrepancy can be roughly tracked back with a factor of about 2 from the Cauchy–Schwarz step of Lemma 4 and about 4 from the worst-case alignment assumed by Weyl's inequality.

Finally (and perhaps most interestingly), on the same data, the output of Algorithm 2 has normalised residual 0.040, well below the threshold of ≈ 0.17. The ICA initialisation therefore lies in a region of certified positive curvature modulo symmetry, giving a second explanation for the speed-up in §3.2.1.

4 Concluding Remarks

We set out to understand why a provably exact algorithm and plain gradient descent behave so differently on the same problem. For the ICA-based method of Bakshi et al. [2019], the answer was that the linear algebra is exact, and error concentrates at a single threshold step. Once that step was treated as a detection problem rather than a tuning problem, it became easier. False positives are asymmetrically expensive, so the optimal threshold is the smallest one limiting them, which can be computed from the ICA output alone.

The comparison with gradient descent is more nuanced. It is far more robust to noise and contamination, and on clean data it tends to reach the same final answer regardless of initialisation. ICA largely helps in speed, with an intuitive geometric reason; ICA gets the 'direction' of hidden units roughly correct (essentially ascertaining the basis vectors of the space spanned by the truth), and that is precisely the degree of freedom gradient descent tries to narrow down.

Our curvature results coincide with this framing. The only flat directions at the truth are the rescalings, the empirical Hessian retains this curvature given enough samples. Most importantly, the ICA initialisation (in select cases) already sits inside the region where our certificate guarantees positive curvature.

Several questions are left open. The certificate holds pointwise and is about ten times conservative, so an interesting area of study would be finding a basin radius in parameter space, which needs Hessian Lipschitz bounds, uniform concentration and more. Our attempts suggest decomposing by hidden unit rather than by layer.

Together with an accuracy guarantee for ICA initialisation, this may lead to an end-to-end exact recovery guarantee for ICA-initialised gradient descent for this particular shallow ReLU model. The thresholding analysis points to an interesting direction of picking multiple thresholds to only enforce constraints within a confidence level, and would be the immediate next step along with tweaking ICA the objective function to reflect the problem specific constraints.

With regards to generalisability of findings to the larger family of shallow models, many of the steps in the ICA-initialisation are contingent on the piecewise linearity of ReLU, and it is unclear if such techniques would be applicable otherwise. ICA itself requires independence sources, which is lost at depth 3 and beyond.

A Derivation of Hessian

A.1 1-Dimensional Restriction

Consider the function $g(t) = f(\mathbf{U} + t\Delta\mathbf{U}, \mathbf{V} + t\Delta\mathbf{V})$, for some arbitrary perturbation directions $\Delta\mathbf{U}$, $\Delta\mathbf{V}$. If $g''(0) \geq 0 \ \forall \ \Delta\mathbf{U}, \Delta\mathbf{V}$, then it must be the case that $\mathbf{H} = \nabla^2 f \succeq 0$. We first try to formulate an expression for $g(t)$ as such:

$$g(t) = \left\|(\mathbf{U} + t \cdot \Delta\mathbf{U})\sigma((\mathbf{V} + t \cdot \Delta\mathbf{V})\mathbf{X}) - \mathbf{A}\right\|_F^2$$

Note that:

$$\begin{aligned}\sigma(\mathbf{VX} + t \cdot \Delta\mathbf{VX}) = \sigma(\mathbf{VX}) + t \cdot \sigma'(\mathbf{VX}) \odot (\Delta\mathbf{VX}) \\ + \frac{t^2}{2} \cdot \sigma''(\mathbf{VX}) \odot (\Delta\mathbf{VX}) \odot (\Delta\mathbf{VX}) + \mathcal{O}(t^3)\end{aligned}$$

So,

$$\begin{aligned}g(t) = \Big\|\mathbf{U}\sigma(\mathbf{VX}) - \mathbf{A} + t \cdot \mathbf{U}\sigma'(\mathbf{VX}) \odot (\Delta\mathbf{VX}) \\ + \frac{t^2}{2} \cdot \mathbf{U}\sigma''(\mathbf{VX}) \odot (\Delta\mathbf{VX}) \odot (\Delta\mathbf{VX}) \\ + t \cdot \Delta\mathbf{U}\sigma(\mathbf{VX}) + t^2 \cdot \Delta\mathbf{U}\sigma'(\mathbf{VX}) \odot (\Delta\mathbf{VX}) + \mathcal{O}(t^3)\Big\|_F^2\end{aligned}$$

For simplicity's sake, rewrite $g(t)$ as such:

$$g(t) = \left\|\mathbf{P}_0 + t \cdot \mathbf{P}_1 + t^2 \cdot \mathbf{P}_2 + \mathcal{O}(t^3)\right\|_F^2,$$

Where

$$\begin{aligned}\mathbf{P}_0 &= \mathbf{U}\,\sigma(\mathbf{VX}) - \mathbf{A} \\ \mathbf{P}_1 &= \mathbf{U}\,\sigma'(\mathbf{VX}) \odot (\Delta\mathbf{VX}) + \Delta\mathbf{U}\,\sigma(\mathbf{VX}) \\ \mathbf{P}_2 &= \tfrac{1}{2}\mathbf{U}\,\sigma''(\mathbf{VX}) \odot (\Delta\mathbf{VX}) \odot (\Delta\mathbf{VX}) + \Delta\mathbf{U}\,\sigma'(\mathbf{VX}) \odot (\Delta\mathbf{VX})\end{aligned}$$

The last equality holding due to the fact that $\sigma'' = 0$ on all but a set of measure zero in the case of ReLU. Using relationship between Frobenius norm and Matrix inner product,

$$g(t) = \left\langle \mathbf{P}_0 + t \cdot \mathbf{P}_1 + t^2 \cdot \mathbf{P}_2 + \mathcal{O}(t^3), \mathbf{P}_0 + t \cdot \mathbf{P}_1 + t^2 \cdot \mathbf{P}_2 + \mathcal{O}(t^3) \right\rangle$$

Expanding in t gives

$$\begin{aligned} g(t) &= \langle \mathbf{P}_0, \mathbf{P}_0 \rangle + 2 \langle \mathbf{P}_0, \mathbf{P}_1 \rangle \cdot t + \left(\langle \mathbf{P}_1, \mathbf{P}_1 \rangle + 2 \langle \mathbf{P}_0, \mathbf{P}_2 \rangle \right) \cdot t^2 + \mathcal{O}(t^3) \\ &= \|\mathbf{P}_0\|_F^2 + 2 \langle \mathbf{P}_0, \mathbf{P}_1 \rangle \cdot t + \left(2 \langle \mathbf{P}_0, \mathbf{P}_2 \rangle + \|\mathbf{P}_1\|_F^2 \right) \cdot t^2 + \mathcal{O}(t^3) \end{aligned}$$

For any twice differentiable g,

$$g(t) = g(0) + g'(0) \cdot t + \frac{g''(0)}{2} \cdot t^2 + O(t^3)$$

Matching coefficients, therefore, gives

$$\begin{aligned} g''(0) &= 4 \langle \mathbf{P}_0, \mathbf{P}_2 \rangle + 2 \|\mathbf{P}_1\|_F^2 \\ &= 4 \left\langle \mathbf{U}\, \sigma(\mathbf{VX}) - A, \Delta\mathbf{U} \sigma'(\mathbf{VX}) \odot (\Delta\mathbf{VX}) \right\rangle \\ &\qquad + 2 \left\| \mathbf{U}\, \sigma'(\mathbf{VX}) \odot (\Delta\mathbf{VX}) + \Delta\mathbf{U}\, \sigma(\mathbf{VX}) \right\|_F^2 \end{aligned}$$

Note that the second term is ≥ 0 by definition, so all of the 'concavity' of loss lies in the term on the left side, which is proportional to the residual $\mathbf{U}\, \sigma(\mathbf{VX}) - \mathbf{A}$.

A.2 Closed form Hessian in terms of Kronecker product

Recall that

$$\begin{aligned} f(\mathbf{U}, \mathbf{V}) &= \|\mathbf{U}\, \sigma(\mathbf{VX}) - \mathbf{A}\|_F^2 \\ &= \langle \mathbf{U}\, \sigma(\mathbf{VX}) - \mathbf{A}, \mathbf{U}\, \sigma(\mathbf{VX}) - \mathbf{A} \rangle \end{aligned}$$

$$\begin{aligned} df &= 2 \langle \mathbf{U}\, \sigma(\mathbf{VX}) - \mathbf{A},\ d\mathbf{U} \cdot \sigma(\mathbf{VX}) \rangle \\ &= 2 \operatorname{tr} \left([\mathbf{U}\, \sigma(\mathbf{VX}) - \mathbf{A}]^\top \cdot d\mathbf{U} \cdot \sigma(\mathbf{VX}) \right) \\ &= 2 \operatorname{tr} \left(\sigma(\mathbf{VX}) [\mathbf{U}\, \sigma(\mathbf{VX}) - \mathbf{A}]^\top \cdot d\mathbf{U} \right) \\ &= 2 \langle [\mathbf{U}\, \sigma(\mathbf{VX}) - \mathbf{A}]\, \sigma(\mathbf{VX})^\top,\ d\mathbf{U} \rangle \\ \implies \frac{\partial f}{\partial \mathbf{U}} &= 2 [\mathbf{U}\, \sigma(\mathbf{VX}) - \mathbf{A}]\, \sigma(\mathbf{VX})^\top \in \mathbb{R}^{m \times k} \end{aligned}$$

Similarly, for the partial derivative with respect to $\mathbf{V}$, we have that:

$$\begin{aligned} df &= 2 \langle \mathbf{U}\, \sigma(\mathbf{VX}) - \mathbf{A}, \mathbf{U} [\sigma'(\mathbf{VX}) \odot (d\mathbf{V} \cdot \mathbf{X})] \rangle \\ &= 2 \langle \mathbf{U}^\top [\mathbf{U}\, \sigma(\mathbf{VX}) - \mathbf{A}], \sigma'(\mathbf{VX}) \odot (d\mathbf{V} \cdot \mathbf{X}) \rangle \\ &= 2 \langle \mathbf{U}^\top [\mathbf{U}\, \sigma(\mathbf{VX}) - \mathbf{A}] \odot \sigma'(\mathbf{VX}), (d\mathbf{V} \cdot \mathbf{X}) \rangle \\ &= 2 \operatorname{tr} \left(\mathbf{U}^\top [(\mathbf{U}\, \sigma(\mathbf{VX}) - \mathbf{A}) \odot \sigma'(\mathbf{VX})]^\top \cdot (d\mathbf{V} \cdot \mathbf{X}) \right) \\ &= 2 \langle \mathbf{U}^\top (\mathbf{U}\, \sigma(\mathbf{VX}) - \mathbf{A}) \odot \sigma'(\mathbf{VX}) \mathbf{X}^\top, d\mathbf{V} \rangle \\ \implies \frac{\partial f}{\partial \mathbf{V}} &= 2 [\mathbf{U}^\top (\mathbf{U}\, \sigma(\mathbf{VX}) - \mathbf{A}) \odot \sigma'(\mathbf{VX})] \mathbf{X}^\top \in \mathbb{R}^{k \times d} \end{aligned}$$

Now we rewrite in terms of Kronecker product $\otimes$. Note the following key properties:

$$\begin{aligned}
\operatorname{vec}(\mathbf{AXB}) &= (\mathbf{B}^\top \otimes \mathbf{A}) \operatorname{vec}(\mathbf{X}) \\
\operatorname{vec}(\mathbf{A} \odot \mathbf{B}) &= \operatorname{diag}(\operatorname{vec}(\mathbf{A})) \operatorname{vec}(\mathbf{B}) \\
(\mathbf{A} \otimes \mathbf{B})(\mathbf{C} \otimes \mathbf{D}) &= (\mathbf{AC}) \otimes (\mathbf{BD}) \\
(\mathbf{A} \otimes \mathbf{B})^\top &= \mathbf{A}^\top \otimes \mathbf{B}^\top \\
\mathbf{K}_{a,b} \operatorname{vec}(\mathbf{P}) &= \operatorname{vec}(\mathbf{P}^\top), \qquad \mathbf{P} \in \mathbb{R}^{a \times b} \\
\mathbf{K}_{a,b}^\top &= \mathbf{K}_{a,b}^{-1} = \mathbf{K}_{b,a}
\end{aligned}$$

$\mathbf{K}_{a,b}$ denotes a commutation (specialised permutation matrix) matrix which converts between vectorisation of transposes. Henceforth $\operatorname{vec}(\cdot)$ refers to column vectorisation. For the first property, when we have multiplication of 2 matrices and we want to express in terms of the vectorisation of one of them, we put in an identity matrix in place of the third.

Using $\mathbf{F} = \sigma(\mathbf{VX})$,

$$\begin{aligned}
\operatorname{vec}\left(\frac{\partial f}{\partial \mathbf{U}}\right) &= \operatorname{vec}\left(2[\mathbf{UF} - \mathbf{A}]\mathbf{F}^\top\right) \\
&= 2 \operatorname{vec}\left(\mathbf{UFF}^\top\right) - 2 \operatorname{vec}\left(\mathbf{AF}^\top\right) \\
&= 2(\mathbf{FF}^\top \otimes \mathbf{I}_m) \operatorname{vec}(\mathbf{U}) - 2 \operatorname{vec}\left(\mathbf{AF}^\top\right) \\
\implies \frac{\partial^2 f}{\partial \operatorname{vec}(\mathbf{U})\, \partial \operatorname{vec}(\mathbf{U})^\top} &= 2(\mathbf{FF}^\top \otimes \mathbf{I}_m) \in \mathbb{R}^{mk \times mk}
\end{aligned}$$

Since the first expression is linear in $\operatorname{vec}(\mathbf{U})$. The second partial w.r.t. $\operatorname{vec}(\mathbf{V})$ is more difficult.

$$\begin{aligned}
\frac{\partial f}{\partial \mathbf{V}} &= 2\left[\mathbf{U}^\top(\mathbf{U}\,\sigma(\mathbf{VX}) - \mathbf{A}) \odot \sigma'(\mathbf{VX})\right]\mathbf{X}^\top \\
\operatorname{vec}\left(\frac{\partial f}{\partial \mathbf{V}}\right) &= 2(\mathbf{X} \otimes \mathbf{I}_k) \operatorname{vec}\left(\mathbf{U}^\top(\mathbf{U}\,\sigma(\mathbf{VX}) - \mathbf{A}) \odot \sigma'(\mathbf{VX})\right) \\
&= 2(\mathbf{X} \otimes \mathbf{I}_k) \operatorname{diag}\left(\operatorname{vec}\left(\sigma'(\mathbf{VX})\right)\right) \operatorname{vec}\left(\mathbf{U}^\top(\mathbf{U}\,\sigma(\mathbf{VX}) - \mathbf{A})\right) \\
&= 2(\mathbf{X} \otimes \mathbf{I}_k) \operatorname{diag}\left(\operatorname{vec}\left(\sigma'(\mathbf{VX})\right)\right) (\mathbf{I}_n \otimes \mathbf{U}^\top) \operatorname{vec}(\mathbf{U}\,\sigma(\mathbf{VX}) - \mathbf{A})
\end{aligned}$$

Letting $\mathbf{R} = \mathbf{U}\,\sigma(\mathbf{VX}) - \mathbf{A}$, $\mathbf{D} = \operatorname{diag}\left(\operatorname{vec}\left(\sigma'(\mathbf{VX})\right)\right)$,

$$\operatorname{vec}\left(\frac{\partial f}{\partial \mathbf{V}}\right) = 2(\mathbf{X} \otimes \mathbf{I}_k)\mathbf{D}(\mathbf{I}_n \otimes \mathbf{U}^\top) \operatorname{vec}(\mathbf{R})$$

Note that:

$$\mathbf{J_V} = \frac{\partial \operatorname{vec}(\mathbf{R})}{\partial \operatorname{vec}(\mathbf{V})^\top} = \frac{\partial \operatorname{vec}(\mathbf{R})}{\partial \operatorname{vec}(\sigma(\mathbf{VX}))^\top} \cdot \frac{\partial \operatorname{vec}(\sigma(\mathbf{VX}))}{\partial \operatorname{vec}(\mathbf{VX})^\top} \cdot \frac{\partial \operatorname{vec}(\mathbf{VX})}{\partial \operatorname{vec}(\mathbf{V})^\top}$$

$$= (\mathbf{I}_n \otimes \mathbf{U}) \cdot \underbrace{\operatorname{diag}\left(\operatorname{vec}\left(\sigma'(\mathbf{VX})\right)\right)}_{:=D} \cdot (\mathbf{X}^\top \otimes \mathbf{I}_k)$$

$$\implies \frac{\partial^2 f}{\partial \operatorname{vec}(\mathbf{V})\ \partial \operatorname{vec}(\mathbf{V})^\top} = 2(\mathbf{X} \otimes \mathbf{I}_k)\mathbf{D}(\mathbf{I}_n \otimes \mathbf{U}^\top)\mathbf{J_V} + 2 \underbrace{\frac{\partial (\mathbf{X} \otimes \mathbf{I}_k)\mathbf{D}(\mathbf{I}_n \otimes \mathbf{U}^\top)}{\partial \operatorname{vec}(\mathbf{V})}}_{=0,\ \text{contains } \sigma''=0 \text{ for ReLU}} \cdot \operatorname{vec}(\mathbf{R})$$

$$= 2(\mathbf{X} \otimes \mathbf{I}_k)\mathbf{D}(\mathbf{I}_n \otimes \mathbf{U}^\top\mathbf{U})\mathbf{D}(\mathbf{X} \otimes \mathbf{I}_k)$$

Finally, for mixed partial derivatives, recall that:

$$\operatorname{vec}\left(\frac{\partial f}{\partial \mathbf{U}}\right) = \frac{\partial f}{\partial \operatorname{vec}(\mathbf{U})} = \operatorname{vec}\left(2[\mathbf{UF} - \mathbf{A}]\mathbf{F}^\top\right) = \operatorname{vec}\left(2\mathbf{RF}^\top\right)$$

To help differentiate w.r.t. $\operatorname{vec}(\mathbf{V})$, denote the perturbation by δ, and try to express everything in terms of $\operatorname{vec}(\delta\mathbf{F})$:

$$\delta(\mathbf{RF}^\top) = (\delta\mathbf{R})\mathbf{F}^\top + \mathbf{R}\delta\mathbf{F}^\top = \mathbf{U}(\delta\mathbf{F})\mathbf{F}^\top + \mathbf{R}\delta\mathbf{F}^\top$$

$$\operatorname{vec}\left(\mathbf{U}(\delta\mathbf{F})\mathbf{F}^\top\right) = (\mathbf{F} \otimes \mathbf{U}) \operatorname{vec}(\delta\mathbf{F})$$

Since $\mathbf{R}\delta(\mathbf{F}^\top)$ contains a transpose, we must use a commutation matrix $\mathbf{K}_{k,n}$:

$$\operatorname{vec}\left(\mathbf{R}\delta\mathbf{F}^\top\right) = (\mathbf{I}_k \otimes \mathbf{R}) \operatorname{vec}\left(\delta\mathbf{F}^\top\right) = (\mathbf{I}_k \otimes \mathbf{R})\mathbf{K}_{k,n} \operatorname{vec}(\delta\mathbf{F})$$

We further have that:

$$\operatorname{vec}(\mathbf{F}) = \operatorname{vec}(\sigma(\mathbf{VX})) = \operatorname{vec}\left(\sigma'(\mathbf{VX}) \odot \mathbf{VX}\right)$$

$$= \operatorname{diag}\left(\operatorname{vec}\left(\sigma'(\mathbf{VX})\right)\right) \operatorname{vec}(\mathbf{VX}) = \mathbf{D}(\mathbf{X}^\top \otimes \mathbf{I}_k) \operatorname{vec}(\mathbf{V})$$

$$\implies \frac{\partial^2 f}{\partial \operatorname{vec}(\mathbf{U})\ \partial \operatorname{vec}(\mathbf{V})^\top} = 2\big[(\mathbf{F} \otimes \mathbf{U}) + (\mathbf{I}_k \otimes \mathbf{R})\mathbf{K}_{k,n}\big]\mathbf{D}(\mathbf{X}^\top \otimes \mathbf{I}_k)$$

Trivially,

$$\frac{\partial^2 f}{\partial \operatorname{vec}(\mathbf{V})\ \partial \operatorname{vec}(\mathbf{U})^\top} = 2\left[[(\mathbf{F} \otimes \mathbf{U}) + (\mathbf{I}_k \otimes \mathbf{R})\mathbf{K}_{k,n}]\mathbf{D}(\mathbf{X}^\top \otimes \mathbf{I}_k)\right]^\top$$

$$= 2(\mathbf{X} \otimes \mathbf{I}_k)\mathbf{D}\left[(\mathbf{F}^\top \otimes \mathbf{U}^\top) + \mathbf{K}_{n,k}(\mathbf{I}_k \otimes \mathbf{R}^\top)\right]$$

Hence concluding our closed-form derivation of the Hessian.

$$\mathbf{H} = \nabla^2 f = \begin{bmatrix} \mathbf{H_{UU}} & \mathbf{H_{UV}} \\ \mathbf{H_{UV}}^\top & \mathbf{H_{VV}} \end{bmatrix}$$

where

$$\begin{aligned}
\mathbf{H_{UU}} &= 2(\mathbf{F}\mathbf{F}^\top \otimes \mathbf{I}_m) \\
\mathbf{H_{UV}} &= 2\left[(\mathbf{F} \otimes \mathbf{U}) + (\mathbf{I}_k \otimes \mathbf{R})\mathbf{K}_{k,n}\right]\mathbf{D}(\mathbf{X}^\top \otimes \mathbf{I}_k) \\
\mathbf{H}_{\mathbf{UV}}^\top &= 2(\mathbf{X} \otimes \mathbf{I}_k)\mathbf{D}\left[(\mathbf{F}^\top \otimes \mathbf{U}^\top) + \mathbf{K}_{n,k}(\mathbf{I}_k \otimes \mathbf{R}^\top)\right] \\
\mathbf{H_{VV}} &= 2(\mathbf{X} \otimes \mathbf{I}_k)\mathbf{D}(\mathbf{I}_n \otimes \mathbf{U}^\top\mathbf{U})\mathbf{D}(\mathbf{X}^\top \otimes \mathbf{I}_k)
\end{aligned}$$

With $\mathbf{F} = \sigma(\mathbf{VX}), \mathbf{R} = \mathbf{UF} - \mathbf{A}, \mathbf{D} = \operatorname{diag}(\operatorname{vec}(\sigma'(\mathbf{VX})))$, as defined earlier.

Observe that there exists a Gauss-Newton decomposition of the above Hessian.

Writing $\mathbf{z} = [\operatorname{vec}(\mathbf{U}); \operatorname{vec}(\mathbf{V})] \in \mathbb{R}^{mk+kd}$, observe the loss is $f = \|\operatorname{vec}(\mathbf{R})\|^2$. Let $\mathbf{J}$ denote the Jacobian of the residual,

$$\mathbf{J} = \frac{\partial \operatorname{vec}(\mathbf{R})}{\partial \mathbf{z}^\top} = \begin{bmatrix}\mathbf{J_U} & \mathbf{J_V}\end{bmatrix} \in \mathbb{R}^{mn\times(mk+kd)},$$

$$\begin{aligned}
\mathbf{J_U} &= \mathbf{F}^\top \otimes \mathbf{I}_m \\
\mathbf{J_V} &= (\mathbf{I}_n \otimes \mathbf{U})\,\mathbf{D}\,(\mathbf{X}^\top \otimes \mathbf{I}_k)
\end{aligned}$$

Then the gradient and Hessian admit the Gauss–Newton decomposition

$$\begin{aligned}
\nabla f &= 2\mathbf{J}^\top \operatorname{vec}(\mathbf{R}) \\
\nabla^2 f &= \underbrace{2\mathbf{J}^\top\mathbf{J}}_{\succeq 0} + \mathbf{S}
\end{aligned}$$

where $\mathbf{S}$ collects the second-order terms arising from differentiating $\mathbf{J}$ itself. These vanish on the diagonal blocks. Can also think of this as writing the Hessian as a multiplication of matrix and its transpose, and $\mathbf{S}$ being the remainder of that. Hence

$$\mathbf{S} = \begin{bmatrix}\mathbf{0} & \mathbf{S_{UV}} \\ \mathbf{S}_{\mathbf{UV}}^\top & \mathbf{0}\end{bmatrix}, \qquad \mathbf{S_{UV}} = 2(\mathbf{I}_k \otimes \mathbf{R})\mathbf{K}_{k,n}\,\mathbf{D}\,(\mathbf{X}^\top \otimes \mathbf{I}_k)$$

Equivalently, as a quadratic form in directions $\Delta\mathbf{U}, \Delta\mathbf{V}$,

$$\begin{aligned}
g''(0) = \mathbf{z}^\top\mathbf{H}\mathbf{z} = &\underbrace{4\langle \mathbf{U}\,\sigma(\mathbf{VX}) - A, \Delta\mathbf{U}[\sigma'(\mathbf{VX}) \odot (\Delta\mathbf{VX})]\rangle}_{=\mathbf{z}^\top\mathbf{S}\mathbf{z}} \\
&+ \underbrace{2\|\mathbf{U}\,\sigma'(\mathbf{VX}) \odot (\Delta\mathbf{VX}) + \Delta\mathbf{U}\,\sigma(\mathbf{VX})\|_F^2}_{=\mathbf{z}^\top\mathbf{2J}^\top\mathbf{J}\mathbf{z}\geq 0 \text{ since } \mathbf{J}^\top\mathbf{J}\succeq 0}
\end{aligned}$$

Since $2\mathbf{J}^\top\mathbf{J} \succeq 0$ always, all indefiniteness in $\nabla^2 f$ arises from $\mathbf{S}$, which is proportional to the residual. Moreover $\mathbf{S}$ is block off-diagonal and traceless, so whenever $\mathbf{S} \neq \mathbf{0}$ it contributes eigenvalues of both signs. Further analysis on $\mathbf{S}$ can show under what conditions our Hessian is indefinite.

B Gauss-Newton Analysis

B.1 Setup

Let $p = mk + kd$ and $\mathrm{z} = \mathrm{vec}\,(\Delta\mathbf{U}, \Delta\mathbf{V}) \in \mathbb{R}^p$. For a single input $\mathrm{x} \in \mathbb{R}^d$, define the per-sample Jacobian $\mathbf{J}(\mathrm{x}) \in \mathbb{R}^{m\times p}$ of the residual by

$$\mathbf{J}(\mathrm{x})\,\mathrm{z} \;=\; \Delta\mathbf{U}\,\sigma(\mathbf{V}\mathrm{x}) \;+\; \mathbf{U}\,\sigma'(\mathbf{V}\mathrm{x}) \odot \Delta\mathbf{V}\mathrm{x}.$$

Equivalently in terms of x_j in Kronecker form:

$$\mathbf{J} = \Big[\sigma(\mathbf{V}\mathrm{x}_j)^\top \otimes \mathbf{I}_m \quad \mathbf{U}\,\mathbf{D}_j(\mathrm{x}_j^\top \otimes \mathbf{I}_k)\Big] \in \mathbb{R}^{m\times p}$$

The empirical Jacobian stacks $\mathbf{J}(\mathrm{x}_1), \ldots, \mathbf{J}(\mathrm{x}_n)$, so that $\mathbf{J}^\top\mathbf{J} = \sum_j \mathbf{J}(\mathrm{x}_j)^\top\mathbf{J}(\mathrm{x}_j)$.

Per-sample, the population Gauss-Newton matrix [Botev, 2020] is $\Sigma^*(\mathbf{U}, \mathbf{V}) \coloneqq \mathbb{E}_{\mathrm{x}\sim\mathcal{N}(\mathbf{0},\mathbf{I}_d)}[\mathbf{J}(\mathrm{x})^\top\mathbf{J}(\mathrm{x})]$. Similarly, denote the empirical Gauss-Newton matrix as $\hat{\mathbf{G}}(\mathbf{U}, \mathbf{V}) \coloneqq \frac{1}{n}\mathbf{J}^\top\mathbf{J} = \frac{1}{n}\sum_{j=1}^n \mathbf{J}(\mathrm{x}_j)^\top\mathbf{J}(\mathrm{x}_j)$, where $\mathrm{x}_1, \ldots, \mathrm{x}_n$ are the columns of $\mathbf{X}$.

We proceed to define the symmetry tangent space as:

$$T_{\mathrm{sym}}(\mathbf{U}, \mathbf{V}) = \{\mathrm{z}_b \coloneqq (\mathbf{U}\,\mathrm{diag}(\mathrm{b}),\; -\mathrm{diag}(\mathrm{b})\,\mathbf{V}) : \mathrm{b} \in \mathbb{R}^k\}.$$

B.2 Ground truth properties for Population Hessian

Proof of Proposition 1. The inclusion $\supseteq$ is Lemma 3. For $\subseteq$, let $\mathrm{z} = (\Delta\mathbf{U}, \Delta\mathbf{V}) \in \ker\Sigma^*$. Then $0 = \mathrm{z}^\top\Sigma^*\mathrm{z} = \mathbb{E}\,\|\mathbf{J}(\mathrm{x})\mathrm{z}\|^2$, so $\mathbf{J}(\mathrm{x})\mathrm{z} = 0$ for almost every x.

Since $\mathbf{V}$ has full row rank and $\mathbf{U}$ has full column rank, we can decompose

$$\Delta\mathbf{V} = \mathbf{A}\mathbf{V} + \Delta\mathbf{V}_\perp, \qquad \Delta\mathbf{V}_\perp\mathbf{V}^\top = 0 \text{ (orthogonal to row space)},$$

$$\Delta\mathbf{U} = \mathbf{U}\mathbf{B} + \Delta\mathbf{U}_\perp, \qquad \mathbf{U}^\top\Delta\mathbf{U}_\perp = 0 \text{ (orthogonal to column space)},$$

for some $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{k\times k}$, splitting each perturbation into a part aligned with the current weights and a part orthogonal to them. Write $\mathrm{g} = \mathbf{V}\mathrm{x}$ and $\mathrm{x} = \mathrm{x}_\parallel + \mathrm{x}_\perp$ with $\mathrm{x}_\parallel \in \mathrm{row}(\mathbf{V})$ and $\mathrm{x}_\perp \perp \mathrm{row}(\mathbf{V})$. Since $\Delta\mathbf{V}_\perp$ annihilates $\mathrm{row}(\mathbf{V})$, we have $\Delta\mathbf{V}\mathrm{x} = \mathbf{A}\mathrm{g} + \Delta\mathbf{V}_\perp\mathrm{x}_\perp$. The activation mask $\mathbf{1}[\mathbf{V}\mathrm{x} > 0] = \mathbf{1}[\mathrm{g} > 0]$ depends on x only through g, so

$$\mathbf{J}(\mathrm{x})\mathrm{z} = (\mathbf{U}\mathbf{B} + \Delta\mathbf{U}_\perp)\,\sigma(\mathrm{g}) + \mathbf{U}(\mathbf{1}[\mathrm{g} > 0] \odot (\mathbf{A}\mathrm{g} + \Delta\mathbf{V}_\perp\mathrm{x}_\perp)). \tag{1}$$

We first show that both orthogonal components vanish. Let $\mathcal{O} = \{\mathrm{x} : \mathbf{V}\mathrm{x} > 0\}$ be the region in which every unit is active. It is open, nonempty because $\mathbf{V}$ is surjective, and therefore has positive probability. On $\mathcal{O}$ we have $\sigma(\mathrm{g}) = \mathrm{g}$ and $\mathbf{1}[\mathrm{g} > 0] = \mathbf{1}$, so (1) becomes

$$\mathbf{J}(\mathrm{x})\mathrm{z} = (\mathbf{U}\mathbf{B} + \Delta\mathbf{U}_\perp)\,\mathrm{g} + \mathbf{U}(\mathbf{A}\mathrm{g} + \Delta\mathbf{V}_\perp\mathrm{x}_\perp). \tag{2}$$

This expression is linear in x, hence continuous; vanishing almost everywhere on the open set $\mathcal{O}$; therefore vanishing on all of $\mathcal{O}$. Left multiplying (2) by $\Delta\mathbf{U}_\perp^\top$, using $\Delta\mathbf{U}_\perp^\top\mathbf{U} = 0$ gives $\Delta\mathbf{U}_\perp^\top\Delta\mathbf{U}_\perp \mathrm{g} = 0 \implies \|\Delta\mathbf{U}_\perp \mathrm{g}\|^2 = 0$ for every $\mathrm{g} \in \mathbf{V}(\mathcal{O})$, which is the open positive orthant of $\mathbb{R}^k$. So $\Delta\mathbf{U}_\perp = 0$.

Next, fix $\mathrm{x}_0 \in \mathcal{O}$ and take any $\mathrm{w} \perp \mathrm{row}(\mathbf{V})$. Then $\mathbf{V}(\mathrm{x}_0 + \mathrm{w}) = \mathbf{V}\mathrm{x}_0$, so $\mathrm{x}_0 + \mathrm{w} \in \mathcal{O}$ with the same g, while $\mathrm{x}_\perp$ (as in (2)) shifts by w. Subtracting (2) at the two points yields $\mathbf{U}\,\Delta\mathbf{V}_\perp \mathrm{w} = 0$ as seen below

$$\begin{aligned} 0 &= \mathbf{J}(\mathrm{x}_0)\mathrm{z} - \mathbf{J}(\mathrm{x}_0 + \mathrm{w})\mathrm{z} \\ &= \big(\mathbf{U}\mathbf{B}\mathrm{g} + \mathbf{U}(\mathbf{A}\mathrm{g} + \Delta\mathbf{V}_\perp \mathrm{x}_{0\perp})\big) - \big(\mathbf{U}\mathbf{B}\mathrm{g} + \mathbf{U}(\mathbf{A}\mathrm{g} + \Delta\mathbf{V}_\perp(\mathrm{x}_{0\perp} + \mathrm{w}))\big) \\ &= -\,\mathbf{U}\,\Delta\mathbf{V}_\perp \mathrm{w}. \end{aligned}$$

As $\mathbf{U}$ is full rank, it follows that $\Delta\mathbf{V}_\perp = 0$.

With both orthogonal components gone, (1) reduces to $\mathbf{J}(\mathrm{x})\mathrm{z} = \mathbf{U}F(\mathbf{V}\mathrm{x})$, where

$$F(\mathrm{g}) = \mathbf{B}\,\sigma(\mathrm{g}) + \mathbf{1}[\mathrm{g} > 0] \odot (\mathbf{A}\mathrm{g}).$$

For each open orthant $\mathcal{O}_s \subset \mathbb{R}^k$, the activation mask is constant within, so on $\mathcal{O}_s$, F is linear in x and by extension so is $\mathbf{U}F$, on the open non-empty set $\{\mathrm{x} : \mathbf{V}\mathrm{x} \in \mathcal{O}_s\}$. Since $\mathbf{U}F$ vanishes almost everywhere, by injectivity of $\mathbf{U}$, it holds that $F = 0$ on every open orthant, i.e. $\{\mathrm{g} \in \mathbb{R}^k : \mathrm{g}_i \neq 0\ \forall i\}$.

We use that particular set of g to remove the discontinuity that arises from stepping over a hyperplane.

Using $\mathbf{1}[\mathrm{g}_i > 0]\,\mathrm{g}_i = \sigma(\mathrm{g}_i)$, row i of $F(\mathrm{g}) = 0$ reads

$$\sum_j \mathbf{B}_{ij}\,\sigma(\mathrm{g}_j) \;+\; \mathbf{1}[\mathrm{g}_i > 0] \sum_j \mathbf{A}_{ij}\,\mathrm{g}_j \;=\; 0.$$

- *Off-diagonal* $\mathbf{B}$. Fix $j \neq i$ and take the orthant with $\mathrm{g}_i < 0$, $\mathrm{g}_j > 0$ and all other coordinates negative. The indicator vanishes and only $\sigma(\mathrm{g}_j)$ survives, so the equation reduces to $\mathbf{B}_{ij}\,\mathrm{g}_j = 0$. Hence $\mathbf{B}_{ij} = 0$ for all $j \neq i$.

- *Off-diagonal* $\mathbf{A}$ *and the diagonal.* Take the orthant with $\mathrm{g}_i > 0$ and all other coordinates negative. Then $\sigma(\mathrm{g}_j) = 0$ for $j \neq i$, and the equation reduces to

  $$(\mathbf{B}_{ii} + \mathbf{A}_{ii})\,\mathrm{g}_i + \sum_{j \neq i} \mathbf{A}_{ij}\,\mathrm{g}_j = 0$$

  for all such g. The left-hand side is linear in g and vanishes on an open set, so every coefficient is zero: $\mathbf{A}_{ij} = 0$ for $j \neq i$ and $\mathbf{B}_{ii} + \mathbf{A}_{ii} = 0$.

Hence, we get $\mathbf{B} = \mathrm{diag}(\mathrm{b})$ and$\mathbf{A} = -\,\mathrm{diag}(\mathrm{b})$, so

$$\Delta\mathbf{U} = \mathbf{U}\,\mathrm{diag}(\mathrm{b}), \qquad \Delta\mathbf{V} = -\,\mathrm{diag}(\mathrm{b})\,\mathbf{V},$$

that is, $\mathrm{z} = \mathrm{z}_b \in T_{\mathrm{sym}}$. □

B.3 Relating Population Hessian to the Empirical Hessian at ground truth

Recall the population Gauss-Newton matrix $\Sigma^* \coloneqq \mathbb{E}_{\mathrm{x}\sim\mathcal{N}(\mathbf{0},\mathbf{I}_d)}\big[\mathbf{J}(\mathrm{x})^\top\mathbf{J}(\mathrm{x})\big]$. and the empirical Gauss-Newton matrix $\hat{\mathbf{G}} \coloneqq \frac{1}{n}\mathbf{J}^\top\mathbf{J} = \frac{1}{n}\sum_{j=1}^n \mathbf{J}(\mathrm{x}_j)^\top\mathbf{J}(\mathrm{x}_j)$ ($\mathbf{U}$, $\mathbf{V}$ implicit as inputs). Denote by $\mathbf{N}$ an orthonormal basis of $T_{\mathrm{sym}}^{\perp}$, so that we can make it simpler by reducing to the complement of the symmetry kernel. Define correspondingly,

$$\mathbf{Y}_j \coloneqq \mathbf{N}^\top\mathbf{J}(\mathrm{x}_j)^\top\mathbf{J}(\mathrm{x}_j)\mathbf{N} \qquad \text{so that} \qquad \mathbf{N}^\top\hat{\mathbf{G}}\,\mathbf{N} = \frac{1}{n}\sum_{j=1}^n \mathbf{Y}_j.$$

We want to be able to bound the minimum eigenvalue of the empirical Gauss-Newton in terms of that of the population, which we shall define as

$$\mu^* \coloneqq \lambda_\perp(\Sigma^*) = \lambda_{\min}(\mathbf{N}^\top\Sigma^*\mathbf{N})$$

where $\lambda_\perp(\cdot)$ is as defined in Definition 1. In order to be able to use a Chernoff bound, we need to have that each $\mathbf{Y}_j \succeq 0$ (holds trivially) and $\lambda_{\max}(\mathbf{Y}_j)$ is bounded above. As $\mathbf{Y}_j$ depends on unbounded $\mathrm{x}_j \sim \mathcal{N}(\mathbf{0},\mathbf{I}_d)$, we use a truncation, and control the tail probability. Observe that $\lambda_{\max}(\mathbf{Y}_j) = \lambda_{\max}\left(\mathbf{N}^\top\mathbf{J}^\top\mathbf{J}\mathbf{N}\right) = \|\mathbf{J}\mathbf{N}\|_2^2 \le \|\mathbf{J}\|_2^2\|\mathbf{N}\|_2^2 = \|\mathbf{J}\|_2^2$, the last equality due to orthonormality of $\mathbf{N}$. We can therefore try to bound $\|\mathbf{J}(\mathrm{x}_j)\|$. We have that:

$$\begin{aligned}
\lambda_{\max}(\mathbf{Y}_j) &\le \|\mathbf{J}(\mathrm{x}_j)\|_2^2 \\
&\le \|\sigma(\mathbf{V}\mathrm{x}_j)^\top \otimes \mathbf{I}_m\|_2^2 + \|\mathbf{U}\mathbf{D}(\mathrm{x}_j)(\mathrm{x}_j^\top \otimes \mathbf{I}_k)\|_2^2 \text{ (decomposing closed form)} \\
&\le \|\sigma(\mathbf{V}\mathrm{x}_j)\|_2^2 + \|\mathbf{U}\|_2^2\|\mathrm{x}_j\|_2^2 \\
&\le \|\mathrm{x}_j\|_2^2 \ (\|\mathbf{U}\|_2^2 + \|\mathbf{V}\|_2^2)
\end{aligned}$$

So x_j is chosen to be truncated to control $\lambda_{\max}(\mathbf{Y}_j)$. Let $\tilde{\mathbf{Y}}_j \coloneqq \mathbf{Y}_j \cdot \mathbf{1}\{\|\mathrm{x}_j\| \le R_{\mathrm{x}}\}$, for some $R_{\mathrm{x}} \in \mathbb{R}_+$. Assuming an appropriate R_{x} is chosen, we can determine the probability of failure within a set of n samples as such:

$$\mathbb{P}(\{\exists j \le n : \|\mathrm{x}_j\| > R_{\mathrm{x}}\}) = \mathbb{P}(\bigcup_{j=1}^n \|\mathrm{x}_j\| > R_{\mathrm{x}}) \le \sum_{j=1}^n \mathbb{P}(\|\mathrm{x}_j\| > R_{\mathrm{x}}),$$

by Boole's inequality. Note that $\|\mathrm{x}_j\|^2 \sim \chi_d^2$, so the RHS of the above equation is simply $n \cdot \mathbb{P}(\chi_d^2 > R_{\mathrm{x}}^2)$. Bounding by R_{x} induces a bias, as $\mathbb{E}\tilde{\mathbf{Y}}_j = \mathbb{E}\mathbf{Y} - \underbrace{\mathbb{E}\big[\mathbf{Y}\cdot\mathbf{1}\{\|\mathrm{x}_j\| > R_{\mathrm{x}}\}\big]}_{=\text{bias}}$. By Lemma 1, we have that $\tilde{\mu} \coloneqq \lambda_{\min}\left(\mathbb{E}\tilde{\mathbf{Y}}\right) \ge \mu^* - \|\text{bias}\|_2$.
So we would want to bound $\|\text{bias}\|_2$ in terms of R_{x}, which would then allow us to use the Chernoff bound. Observe that

$$\Big\|\mathbb{E}\left[\mathbf{Y}\cdot\mathbf{1}\{\|\mathrm{x}\| > R_{\mathrm{x}}\}\right]\Big\|_2 \le \mathbb{E}\Big[\|\mathbf{Y}\|_2 \cdot \mathbf{1}\{\|\mathrm{x}\| > R_{\mathrm{x}}\}\Big], \tag{3}$$

due to Jensen's inequality with convexity of $\|\cdot\|_2$ (all norms convex due to triangle inequality and homogeneity). Recall that $\|\mathbf{Y}\|_2 \le \|\mathbf{J}(\mathrm{x})\|_2^2 \le c \cdot \|\mathrm{x}\|^2$, where $c \coloneqq (\|\mathbf{U}\|_2^2 + \|\mathbf{V}\|_2^2)$. To bound the RHS of 3, - let $Q \coloneqq \|\mathrm{x}\|^2 \sim \chi_d^2$

$$\begin{aligned}
\mathbb{E}\Big[\|\mathrm{x}\|^2 \cdot \mathbf{1}\{\|\mathrm{x}\| > R_{\mathrm{x}}\}\Big] &= \mathbb{E}\Big[Q \cdot \mathbf{1}\{Q > R_{\mathrm{x}}^2\}\Big] \\
&= \int_{R_{\mathrm{x}}^2}^{\infty} q \cdot \frac{1}{2^{d/2}\Gamma(d/2)} q^{d/2-1} e^{-q/2} dq \\
&= d \int_{R_{\mathrm{x}}^2}^{\infty} \frac{1}{2^{(d+2)/2}\,\Gamma(\frac{d+2}{2})}\, q^{(d+2)/2-1} e^{-q/2}\, dq \\
&= d\; \mathbb{E}[\mathbf{1}\{\chi_{d+2}^2 > R_{\mathrm{x}}^2\}] \\
&= d\; \mathbb{P}(\chi_{d+2}^2 > R_{\mathrm{x}}^2)
\end{aligned}$$

Putting everything together, we have that

$$\mathbb{E}\big[\mathbf{Y} \cdot \mathbf{1}\{\|\mathrm{x}\| > R_{\mathrm{x}}\}\big] \le c \cdot d \cdot \mathbb{P}(\chi_{d+2}^2 > R_{\mathrm{x}}^2)$$

We want the LHS to be arbitrarily $\le \frac{\mu^*}{4}$ (for further bounding purposes), so we pick R_{x} such that

$$\mathbb{P}(\chi_{d+2}^2 > R_{\mathrm{x}}^2) \le \frac{\mu^*}{4dc}$$

The cumulative distribution function of a χ^2 random variable has no closed form analytical inverse, but one can use the following lemma to bound its tail probabilities.

Lemma 5 (Laurent–Massart Concentration Bounds)**.** *Let $Q \sim \chi_D^2$. Then for all $t > 0$,*

$$\mathbb{P}(Q \ge D + 2\sqrt{Dt} + 2t) \le e^{-t}, \qquad \mathbb{P}(Q \le D - 2\sqrt{Dt}) \le e^{-t}.$$

Proceeding, set

$$t \coloneqq \log \max\Big\{\frac{4dc}{\mu^*},\ \frac{2n}{\delta}\Big\}$$

and choose

$$R_{\mathrm{x}}^2 \coloneqq (d+2) + 2\sqrt{(d+2)\,t} + 2t.$$

By Lemma 5 with $D = d + 2$,

$$\mathbb{P}(\chi_{d+2}^2 > R_{\mathrm{x}}^2) \le e^{-t} \le \min\Big\{\frac{\mu^*}{4dc},\ \frac{\delta}{2n}\Big\}.$$

Hence

$$\mathbb{E}\big[\|\mathrm{x}\|^2\, \mathbf{1}\{\|\mathrm{x}\| > R_{\mathrm{x}}\}\big] = d\, \mathbb{P}(\chi_{d+2}^2 > R_{\mathrm{x}}^2) \le \frac{\mu^*}{4c},$$

and so the truncation bias is at most $c \cdot \frac{\mu^*}{4c} = \frac{\mu^*}{4}$.

Connecting back, recall the truncated matrices $\tilde{\mathbf{Y}}_j = \mathbf{Y}_j \cdot \mathbf{1}\{\|\mathrm{x}_j\| \le R_{\mathrm{x}}\}$. These are independent and positive semidefinite, and satisfy $\lambda_{\max}\left(\tilde{\mathbf{Y}}_j\right) \le R_{\mathrm{x}}^2 c =: R$.

$$\mathbb{E}\tilde{\mathbf{Y}}_j = \mathbb{E}\mathbf{Y} - \mathbb{E}\big[\mathbf{Y} \cdot \mathbf{1}\{\|\mathrm{x}_j\| > R_{\mathrm{x}}\}\big]$$

So by Lemma 1,

$$\tilde{\mu} \coloneqq \lambda_{\min}\left(\mathbb{E}\,\tilde{\mathbf{Y}}_j\right) \geq \mu^* - \frac{\mu^*}{4} = \frac{3}{4}\mu^*.$$

Applying Lemma 2 to $\sum_{j=1}^n \tilde{\mathbf{Y}}_j$ in dimension $p-k$ with $\varepsilon = 1/3$,

$$\mathbb{P}\Big(\lambda_{\min}\left(\sum_j \tilde{\mathbf{Y}}_j\right) \leq \tfrac{2}{3}\, n\tilde{\mu}\Big) \leq (p-k)\exp\Big(-\frac{n\tilde{\mu}}{18R}\Big) \leq (p-k)\exp\Big(-\frac{n\mu^*}{24R}\Big),$$

which is at most $\delta/2$ whenever

$$n \geq \frac{24R}{\mu^*}\log\frac{2(p-k)}{\delta}.$$

Moreover, since $\chi^2_{d+2} \overset{d}{=} \chi^2_d + \chi^2_2$ with the second term nonnegative and independent, $\mathbb{P}(\chi^2_d > r) \leq \mathbb{P}(\chi^2_{d+2} > r)$ for all r. A union bound over the samples (Boole's inequality):

$$\mathbb{P}(\exists j \leq n : \|\mathrm{x}_j\| > R_{\mathrm{x}}) \leq n\,\mathbb{P}(\chi^2_d > R^2_{\mathrm{x}}) \leq n\,\mathbb{P}(\chi^2_{d+2} > R^2_{\mathrm{x}}) \leq \frac{\delta}{2}.$$

On the complement of both events, $\tilde{\mathbf{Y}}_j = \mathbf{Y}_j$ for every j, so $\sum_j \tilde{\mathbf{Y}}_j = n\,\mathbf{N}^\top\hat{\mathbf{G}}\mathbf{N}$. Therefore, with probability at least $1-\delta$,

$$\lambda_\perp(\hat{\mathbf{G}}) \geq \tfrac{2}{3}\cdot\tfrac{3}{4}\,\mu^* = \frac{\mu^*}{2}.$$

We have bounded the minimum eigenvalue of the empirical Gauss-Newton with respect to that of the population Gauss-Newton. R grows logarithmically with respect to n, condition holds with high probability.

B.4 Bounding $\|\mathbf{S}\|_2$

Recall the Gauss-Newton Decomposition from earlier, $\nabla^2 f = 2\mathbf{J}^\top\mathbf{J} + \mathbf{S}$, where

$$\mathbf{S} = \begin{bmatrix} \mathbf{0} & \mathbf{S_{UV}} \\ \mathbf{S_{UV}}^\top & \mathbf{0} \end{bmatrix}, \qquad \mathbf{S_{UV}} = 2(\mathbf{I}_k \otimes \mathbf{R})\mathbf{K}_{k,n}\,\mathbf{D}\,(\mathbf{X}^\top \otimes \mathbf{I}_k)$$

$\mathbf{S}$ is trivially 0 at the ground truth due to the presence of residual term $\mathbf{R}$. We can however still bound the minimum eigenvalue of the Hessian where $\mathbf{U} \neq \mathbf{U}^*, \mathbf{V} \neq \mathbf{V}^*$ by applying Weyl's inequality to the Hessian projected onto the quotient space of the symmetry tangent to ignore the k eigenvalues of 0.

As a consequence of Lemma 1, observe that

$$\lambda_\perp\left(\nabla^2 f\right) \geq 2\lambda_\perp\left(\mathbf{J}^\top\mathbf{J}\right) - \|\mathbf{S}\|_2. \tag{4}$$

We can now bound $\|\mathbf{S}\|_2$ by analysing its quadratic form (recall $z = \operatorname{vec}(\Delta\mathbf{U}, \Delta\mathbf{V}) \in \mathbb{R}^p$):

$$\begin{aligned}
|z^\top \mathbf{S} z| &= 4\left|\langle \mathbf{R},\ \Delta\mathbf{U}[\mathbf{1}[\mathbf{V}\mathbf{X} > 0] \odot (\Delta\mathbf{V}\mathbf{X})]\rangle\right| \\
&\le 4\,\|\mathbf{R}\|_F \left\|\Delta\mathbf{U}[\mathbf{1}[\mathbf{V}\mathbf{X} > 0] \odot (\Delta\mathbf{V}\mathbf{X})]\right\|_F \\
&\le 4\,\|\mathbf{R}\|_F \underbrace{\|\Delta\mathbf{U}\|_F \,\|\Delta\mathbf{V}\|_F}_{\le \frac{1}{2}(\|\Delta\mathbf{U}\|_F^2 + \|\Delta\mathbf{V}\|_F^2) = \frac{1}{2}\|z\|^2} \|\mathbf{X}\|_2 \\
&\le 2\,\|\mathbf{R}\|_F\,\|\mathbf{X}\|_2\,\|z\|^2,
\end{aligned}$$

so $\|\mathbf{S}\|_2 \le 2\|\mathbf{R}\|_F\|\mathbf{X}\|_2$. Substituting into (4),

$$\lambda_\perp\left(\nabla^2 f\right) \ge 2\lambda_\perp\left(\mathbf{J}^\top\mathbf{J}\right) - 2\|\mathbf{R}\|_F\,\|\mathbf{X}\|_2,$$

which is positive whenever

$$\|\mathbf{R}\|_F < \frac{\lambda_\perp\left(\mathbf{J}^\top\mathbf{J}\right)}{\|\mathbf{X}\|_2} \quad\Longleftrightarrow\quad \frac{\|\mathbf{R}\|_F}{\sqrt{n}} < \frac{\lambda_\perp\left(\frac{1}{n}\mathbf{J}^\top\mathbf{J}\right)}{\|\mathbf{X}\|_2/\sqrt{n}}.$$

Since $\mathbf{X}$ has i.i.d. $\mathcal{N}(0,1)$ entries, with probability at least $1 - 2e^{-t^2/2}$ we have $\|\mathbf{X}\|_2 \le \sqrt{n} + \sqrt{d} + t$ [Vershynin, 2018, Corollary 7.3.3], so it suffices that

$$\frac{\|\mathbf{R}\|_F}{\sqrt{n}} < \frac{\lambda_\perp\left(\frac{1}{n}\mathbf{J}^\top\mathbf{J}\right)}{1 + \frac{\sqrt{d}+t}{\sqrt{n}}}.$$

for $\nabla^2 f$ on the quotient space to be positive definite on $T_{\mathrm{sym}}^\perp$.

References

Sanjeev Arora, Rong Ge, Ankur Moitra, and Sushant Sachdeva. Provable ICA with unknown Gaussian noise, with implications for Gaussian mixtures and autoencoders. In *Advances in Neural Information Processing Systems*, volume 25, pages 2375–2383, 2012.

Ainesh Bakshi, Rajesh Jayaram, and David P. Woodruff. Learning two layer rectified neural networks in polynomial time. In *Conference on Learning Theory*, pages 195–268, 2019.

Aleksandar Botev. *The Gauss–Newton Matrix for Deep Learning Models and its Applications*. PhD thesis, University College London, 2020.

Saikiran Bulusu, Geethu Joseph, M. Cenk Gursoy, and Pramod K. Varshney. Learning distributions generated by single-layer relu networks in the presence of arbitrary outliers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Pierre Comon. Independent component analysis, a new concept? *Signal Processing*, 36(3):287–314, 1994.

Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, 2nd edition, 2012.

Aapo Hyvärinen. Fast and robust fixed-point algorithms for independent component analysis. *IEEE Transactions on Neural Networks*, 10(3):626–634, 1999.

Harold W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2:83–97, 1955.

Kaare Brandt Petersen and Michael Syskind Pedersen. The matrix cookbook, 2012. Version 20121115.

Itay Safran and Ohad Shamir. Spurious local minima are common in two-layer ReLU neural networks. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *PMLR*, pages 4433–4441, 2018.

Joel A. Tropp. User-friendly tail bounds for sums of random matrices. *Foundations of Computational Mathematics*, 12(4):389–434, 2012.

Roman Vershynin. *High-Dimensional Probability: An Introduction with Applications in Data Science*. Cambridge University Press, 2018.

Kai Zhong, Zhao Song, Prateek Jain, Peter L. Bartlett, and Inderjit S. Dhillon. Recovery guarantees for one-hidden-layer neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *PMLR*, pages 4140–4149, 2017.